Gemini 3.8 Flash レイテンシ・アーキテクチャ評価レポート

Google AI Studio（`google-genai` SDK）における Streaming 配信・`ThinkingConfig`・Pydantic 適応型ルーティング・50ターン暗黙キャッシュの実機検証

Author: Yuichi Ito | **Date:** 2026-11 | **Target:** Google AI Studio (`google-genai` SDK / `gemini-3.8-flash`)



図0-1: Google AI Studio (gemini-3.8-flash) におけるレイテンシ・アーキテクチャ検証の3大評価軸と主要結論

レポート概要と章構成（全5章）

本レポートは、Google AI Studio（`google-genai` SDK、`genai.Client(api_key=GEMINI_KEY)`）上の `gemini-3.8-flash`（および Gemini 3.5 / 4 シリーズ全5モデル）を対象に、パソコンが動かなくなったトラブルシュートを行う顧客とサポート窓口の対話シナリオを通じて、対話型AIアプリケーションの初期応答速度（TTFT：Time to First Token）と深層推論品質・長期対話スケーラビリティを両立するためのアーキテクチャ設計を、実機 API 計測ログ（`src/gemini_evaluation/`）に基づいて体系化した技術白書である。

章番号	タイトル	検証内容と主要インサイト
第1章	**エグゼクティブサマリー**	全検証の結論マトリクス、主要ベンチマーク指標の横断比較、および本番システムに向けた推奨アーキテクチャ設計指針。
第2章	**前提知識と測定方法**	LLM推論の3フェーズ（Prefill・Thinking・Decoding）、2次元トラジェクトリ（経過時間×累積出力文字数）、Pydantic `response_schema` の Cold/Warm 文法コンパイル特性と `pydantic_core.from_json(..., allow_partial="trailing-strings")` による逐次差分抽出、`ExecutionTracer` の計測基盤、および **Gemini 3.5 / 3.8 / 4 シリーズ全モデルの分類と `ThinkingConfig`・Context Caching 仕様の実機比較検証**。
第3章	**パラメータとレイテンシー**	Blocking（`generate_content`）vs. Streaming（`generate_content_stream`）× `ThinkingConfig` 全6段階の実測比較、内部推論トークン数と TTFT の線形スケーリング則、PCサポート難易度3段階（Tier 1〜3）におけるトレードオフ、および **現行5モデル × 全 `thinking_level`（17構成）＋非推奨 `thinking_budget=0`（5構成）の計22構成×10問（220試行）横断レイテンシー比較**。
第4章	**適応型 Thinking ルーター（Adaptive Thinking Router）**	Stage 1（簡易判断）と Stage 2（Thinking 推論：用途別3カテゴリ A/B/C）のモデル・`thinking_level` 選定候補と、`gemini-3.8-flash`（`low → high`）による45試行比較。平均 TTFT `2,336 ms`・分岐精度 `100%`・複雑計算 `100%` を達成する **Pattern A** の優位性。
第5章	**マルチターンチャットでの挙動とコンテキストキャッシュ**	全構成を **Streaming に統一** し、50ターン連続PCサポート対話（計150実行）において `(1)` `3.8-flash` `low` / `high` と `(2)` **第4章の Adaptive Thinking 構成（Pattern A `low → high`）** の初期応答（TTFT）と完了時間（Total）、および Stage 1→2 間で自動共有される暗黙的コンテキストキャッシュ（最大 `90.2%` ヒット）を実証。

ドキュメントメタデータ	内容
タイトル（`title`）	Gemini 3.8 Flash レイテンシ・アーキテクチャ評価レポート
サブタイトル（`subtitle`）	Google AI Studio における Streaming・ThinkingConfig・適応型ルーティング・50ターン対話の実機検証
著者（`author`）	Yuichi Ito
作成年月（`date`）	2026-11
検証 SDK / モデル	Google AI Studio `google-genai` SDK（`genai.Client(api_key=GEMINI_KEY)`）/ `gemini-3.8-flash` ほか現行5モデル
計測コード・ログ	`src/gemini_evaluation/llm_call.py`, `src/gemini_evaluation/adaptive_thinking.py`, `src/gemini_evaluation/tracer.py`

目次

1. エグゼクティブサマリー

2. 前提知識と測定方法

3. パラメータとレイテンシー

4. Adaptive Thinking Router パターン

5. マルチターンチャットでの挙動とコンテキストキャッシュ

1. エグゼクティブサマリー

本レポートは、Google AI Studio（`google-genai` SDK、`genai.Client(api_key=GEMINI_KEY)`）上の `gemini-3.8-flash`（および現行5モデル）を用いた対話型AIシステム（PCテクニカルサポート・トラブルシューティング対話）において、**「高速な初期応答（TTFT）」「多段階算数・条件比較（修理vs買い替え損益・データ転送時間・保守契約試算）の100%の計算正確性」「50ターン連続対話における平坦なレイテンシスケーリング」** の3要件を同時に達成するためのアーキテクチャ検証報告書である。実機 API 計測ログ（単発 `ThinkingConfig` 18試行、現行5モデル×全 `thinking_level` ＋ `budget=0` 横断比較 220試行、適応型ルーティング `low → high` 45試行、50ターン連続PCサポート Streaming チャット 150試行）の定量分析から、単発・マルチターンを問わず `Streaming`（`generate_content_stream`）**を基本通信方式とし、Stage 1（`low`）の Pydantic 2フィールド構造化ストリーム `{thinking: bool, response: str}` と Stage 2（`high`）の並列起動を組み合わせた「適応型 Thinking ルーター（Pattern A）」を標準採用する構成** を第一推奨アーキテクチャとして結論づけた。

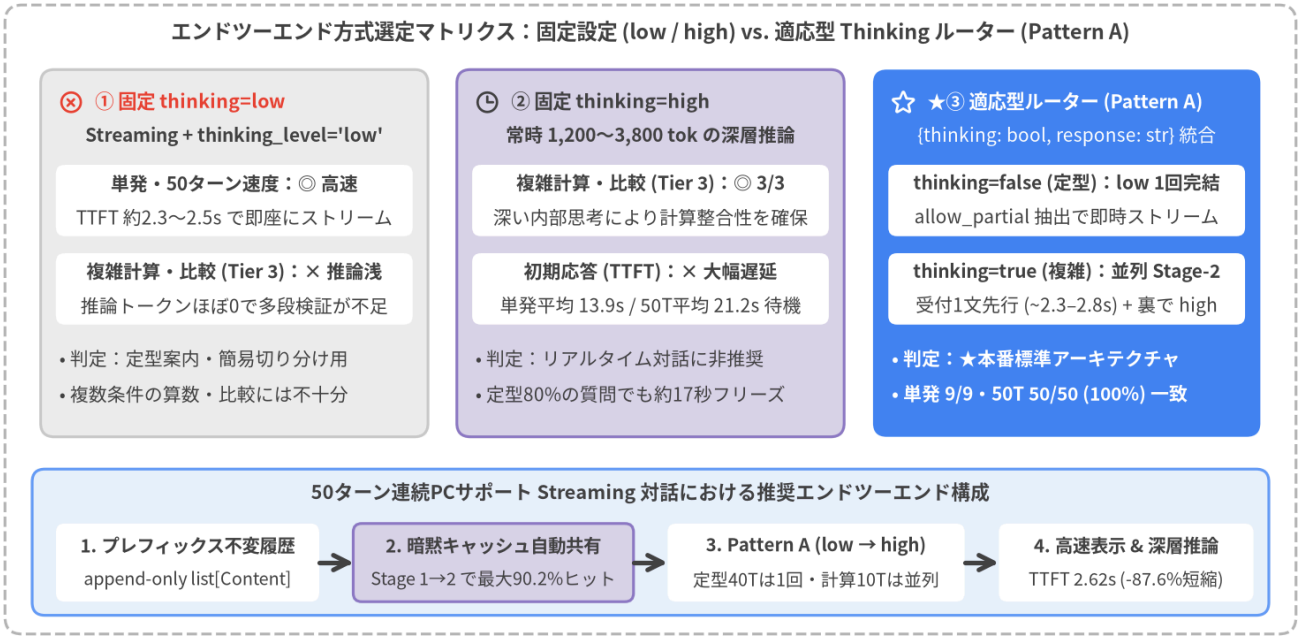


図1-1: エンドツーエンド方式選定マトリクス – 固定設定（low / high）の二律背反と適応型ルーター（Pattern A）＋50ターン暗黙キャッシュ統合構成

1.1 全検証スイートの主要ベンチマーク横断比較

本レポートで実施した3つのコア評価スイート（① 単発通信方式 × `ThinkingConfig` ベンチマーク、② 適応型 Thinking ルーター `low → high` 評価、③ 50ターン連続マルチターンPCサポート Streaming 対話ベンチマーク）の代表レイテンシ指標（初期応答 `TTFT` および完了時間 `Total`）を横断比較したのが図1-2である。

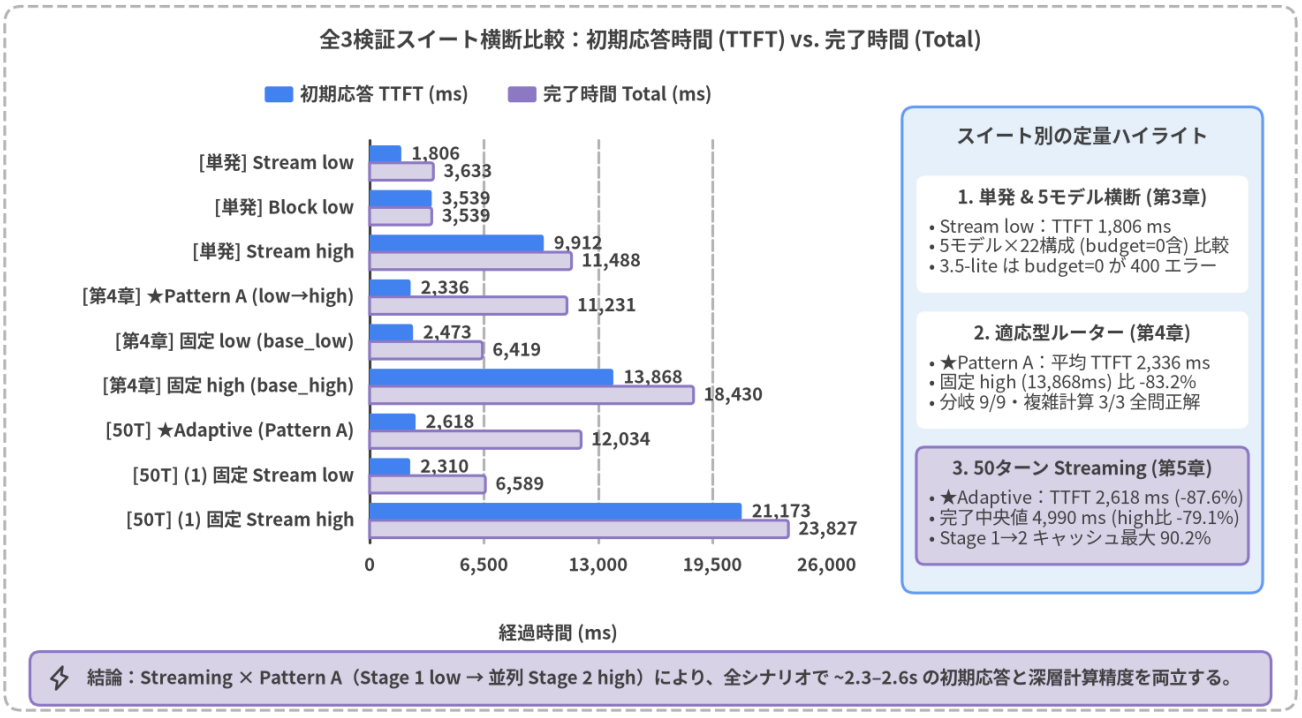


図1-2: 全3検証スイート（単発呼び出し・適応型ルーター・50ターン連続 Streaming 対話）における代表構成の初期応答 (TTFT) と完了時間 (Total) の横断比較

全検証スイート横断サマリー表

評価スイート・シナリオ	方式・パラメータ構成	初期応答 (TTFT)	完了時間 (Total)	推論トークン (`ThTok`)	品質・スケーラビリティ評価	判定
単発呼び出し (第3章・PC起動切り分け)	**Streaming** `thinking_level="low"`	`1,806 ms`	`3,633 ms`	`135 tok`	初回表示が Blocking 比 `-49%` 高速（`minimal` なら `1,304 ms`）	**★定型推奨**
単発呼び出し	Blocking `thinking_level="low"`	`3,539 ms`	`3,539 ms`	`124 tok`	全文完了まで `3.54秒` 画面無応答	バッチ用
単発呼び出し	Streaming `thinking_level="high"`	`9,912 ms`	`11,488 ms`	`946 tok`	内部推論により単発でも TTFT が約10秒へ増大	単発固定は非推奨

評価スイート・シナリオ	方式・パラメータ構成	初期応答 (TTFT)	完了時間 (Total)	推論トークン (ThTok)	品質・スケーラビリティ評価	判定
5モデル横断比較 (第3章・22構成×10問)	`3.5-flash-lite` / `3.8-flash` / `4` 系 (`thinking_level` + 非推奨 `budget=0`)	`706 ms` 〜 `30,368 ms`	`2,240 ms` 〜 `34,637 ms`	`0` 〜 `2,369 tok`	`3.5-flash-lite` は `budget=0` が `400 INVALID_ARGUMENT` (`minimal` 必須)。他4モデルは `low` 相当へフォールバック	`thinking_level` へ統一
適応型ルーター (第4章・`3.8-flash` `low→high`)	**★Pattern A (`{thinking, response}`)**	`2,336 ms` (T3: `2,337 ms`)	`11,231 ms` (T1・2: `6,625 ms`)	T1・2: `95` T3: `1,392 tok`	分岐 `9/9`・Tier 3 算数 `3/3` **(100%)** 固定 `high` 比で初期応答 `-83.2%` 短縮	**★本番標準**
適応型ルーター比較	Pattern C (先頭タグ `[DIRECT]/[THINK]`)	`2,063 ms`	`10,167 ms`	T1・2: `66` T3: `1,197 tok`	分岐 `9/9`・計算 `3/3`。ただし構文保証なし・タグ露出リスクあり	不採用
適応型ルーター比較	Pattern B (1項目分離 `{thinking}`)	`6,770 ms` (T3: `13,865 ms`)	`11,648 ms`	T1・2: `172` T3: `1,355 tok`	常時2回直列呼び出し (`calls=2.0`) により初動が大幅遅延	不採用
適応型ルーター比較	固定 `thinking_level="low"` (`base_low`)	`2,473 ms`	`6,419 ms`	平均 `91 tok`	高速だが Tier 3 複雑計算で深い推論が行われない	複雑計算に不適
適応型ルーター比較	固定 `thinking_level="high"` (`base_high`)	`13,868 ms` (T3: `20,037 ms`)	`18,430 ms`	平均 `1,201 tok`	Tier 3 計算は深いが、定型質問でも `9.8〜20.0秒` 画面が停止	固定運用は非推奨
50ターン Streaming (第5章・定型40T＋計算10T)	**★ `(2) adaptive_pat_a` (Pattern A)**	`2,618 ms` (中央値 `2,420 ms`)	`12,034 ms` (中央値 `4,990 ms`)	平均 `758 tok` (`high` 比 `-61.7%`)	分岐 `50/50 (100%)`。定型40Tは `5,008 ms` 完結、計算10Tは並列 `high` (キャッシュ最大 `90.2%` 共有)	**★長期対話標準**
50ターン Streaming	`(1) streaming_low` (`low` 固定)	`2,310 ms` (中央値 `2,084 ms`)	`6,589 ms` (中央値 `6,583 ms`)	`0 tok`	全ターン高速だが、10回の複雑計算ターンでも推論トークン `0`	定型専用
50ターン Streaming	`(1) streaming_high` (`high` 固定)	`21,173 ms` (計算10T: `38,033 ms`)	`23,827 ms` (中央値 `23,887 ms`)	平均 `1,981 tok`	定型40ターンでも平均 `16,958 ms` (`1,534 tok`) 初動がフリーズ	非推奨

1.2 本番システム設計に向けた5つの推奨アクション

実機検証から導かれた、Google AI Studio (`gemini-3.8-flash` および Gemini 3.5 / 4 シリーズ) を本番組み込みする際のアーキテクチャ設計原則を以下の5点に要約する。

1. **ユーザー向け対話 UI はすべて `generate_content_stream`（Streaming）に統一する**： 単発呼び出しでも Blocking `low`（`3,539 ms`）に対して Streaming `low`（`TTFT 1,806 ms`）は `-49%` 早く表示を開始し、50ターン連続対話でも初期応答を約 `2.3〜2.6 秒` に抑えながら残りの生成時間をユーザーの読解時間の裏へ隠蔽できる。
2. **非推奨の `thinking_budget=0` ではなく、`thinking_level`（`minimal` / `low` / `medium` / `high`）で明示制御する**： 第3章の5モデル横断検証（220試行）で実証した通り、レガシーな `thinking_budget=0` は `gemini-3.5-flash-lite` で `400 INVALID_ARGUMENT` エラーとなり、他4モデルでも内部的に `low` 相当へフォールバックされる。ゼロ推論が必要な場合は `thinking_level="minimal"`（`3.5-flash-lite` / `3.8-flash`）、軽量ルーティングには `thinking_level="low"` を明示指定すべきである。
3. **「`low` と `high` の二律背反」は第4章の Pattern A（Pydantic 2項目統合ルーター `low → high`）で解決する**： `low` 固定では複雑計算ターンで深い推論が行われず、`high` 固定では単発平均 `13.9 秒`・50ターン平均 `21.2 秒` も画面が停止する。`AdaptiveStreamResponse(thinking: bool, response: str)` を Stage 1（`3.8-flash [low]`）でストリームしつつ `thinking=True` 検知時に Stage 2（`3.8-flash [high]`）を並列起動する **Pattern A** により、全50ターン平均 TTFT `2,618 ms`（`high` **比** `-87.6%`）・完了中央値 `4,990 ms`（`-79.1%`）・分岐一致 `50/50 (100%)` を達成できる。
4. **要件に応じて Stage 1 を `3.5-flash-lite`、Stage 2 を `Gemini 4` シリーズへスケールさせる**： 第4章（4.1節）で整理した通り、初期応答をさらに短縮（`~1.0〜1.5s`）したい場合は Stage 1 に `gemini-3.5-flash-lite`（`minimal` / `low`）を採用し、複雑な業務システムでより深い推論が必要な場合は Stage 2 に `gemini-4-hydrogen [high]` や `gemini-4-carbon [medium / high]` を組み合わせる構成が推奨される。
5. **マルチターン対話では履歴プレフィックスを不変（append-only）に保ち、Stage 1→2 間で暗黙キャッシュを共有する**： `system_instruction` を Stage 1・Stage 2 間で完全一致させ、`history: list[types.Content]` を追記専用（append-only）に保つことで、ターン8〜9（約2,710トークン）から **Implicit Context Caching**（`cached_content_token_count`）が自動発動し、複雑計算ターンの Stage 2（`high`）でも最大 `90.2%`（**ターン50で** `18,890 cached tokens`）のキャッシュヒット率を享受できる。

2. 前提知識と測定方法

本章では、第3章以降の実機ベンチマーク結果を正確に読み解くための技術的な前提知識と計測メソドロジーを定義する。前半（2.1〜2.2節）では、`gemini-3.8-flash` の推論レイテンシを構成する3つの内部フェーズ（**Prefill・Thinking・Decoding**）と、単一の平均値では捉えられない体感差を可視化する **2次元トラジェクトリ（経過時間 × 累積出力文字数）**、および Google AI Studio（`google-genai` SDK）における Blocking / Streaming 実装パターンを解説する。後半（2.3〜2.5節）では、適応型 Thinking ルーターの基盤となる **Pydantic 構造化出力（`response_schema`）の Cold/Warm 文法コンパイル特性** と `pydantic_core.from_json(..., allow_partial="trailing-strings")` **による不完全 JSON 逐次差分抽出**、マイクロ秒精度で全イベントを記録する `ExecutionTracer`（`src/gemini_evaluation/tracer.py`）の設計、そして **現行 Gemini 3.5 / 3.8 / 4 シリーズ全モデルの分類と** `ThinkingConfig` **・Context Caching 仕様の実機比較検証** を詳述する。

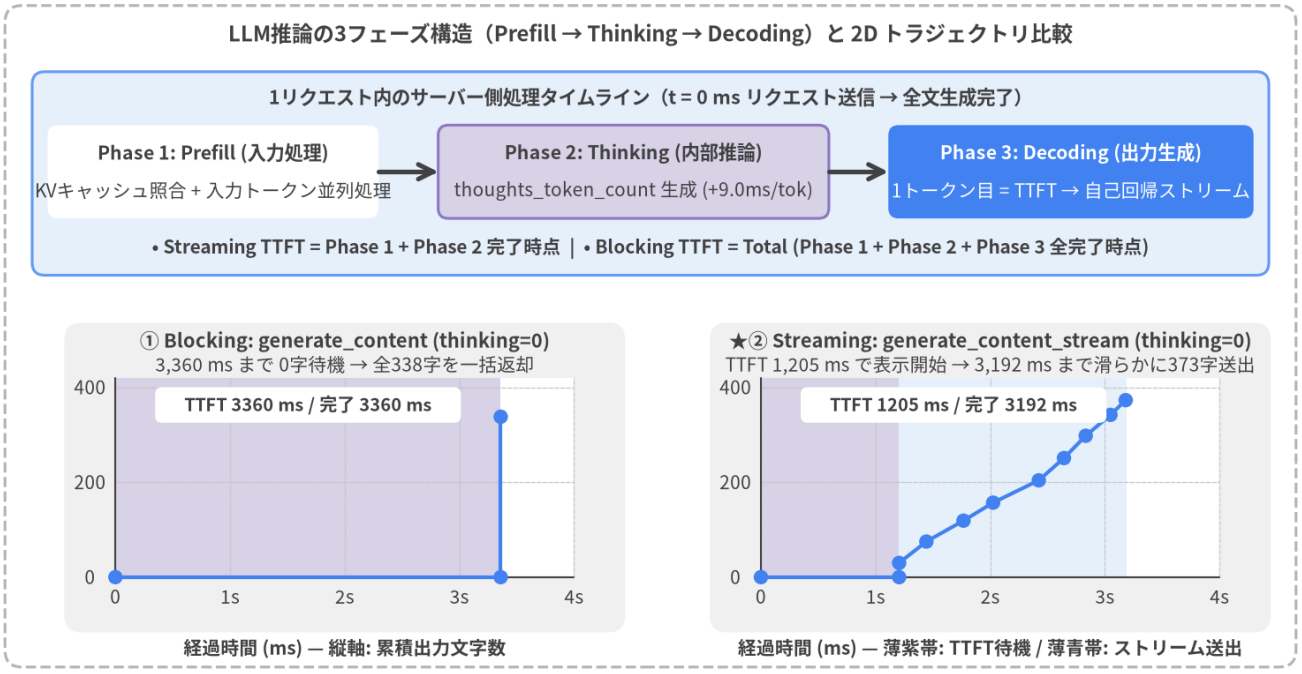


図2-1: LLM推論の3フェーズ構造（Prefill → Thinking → Decoding）と Blocking vs. Streaming の2次元トラジェクトリ比較（thinking=0 実測値）

2.1 レイテンシの定義（3フェーズ構造）と2次元トラジェクトリ解剖

大規模言語モデル（LLM）の API 呼び出しにおける応答遅延を正確に評価・最適化するためには、単なる「リクエストから完了までの合計時間」ではなく、サーバー内部で直列に進行する **3つの処理フェーズ** に分解して理解する必要がある。

1. LLM 推論を構成する 3 つのフェーズ

1. **Phase 1: Prefill（入力処理＋KVキャッシュ照合）** ： クライアントから送信された `system_instruction`、会話履歴 `history`、および最新のユーザー入力 `prompt`（合計 `prompt_token_count`）を Transformer レイヤーで並列処理し、Attention の Key-Value（KV）キャッシュを構築するフェーズである。Google AI Studio の `gemini-3.8-flash` では、過去の対話履歴プレフィックスがキャッシュにヒット（`cached_content_token_count`）した場合、該当トークン分の Prefill 計算がスキップされ、数万トークンの長大コンテキストでも初動遅延が最小化される（第5章で詳述）。
2. **Phase 2: Thinking（内部推論トークン生成：`thoughts_token_count`）** ： `gemini-3.8-flash` のような推論（Reasoning / Thinking）対応モデルにおいて、ユーザーに見える回答本文（`chunk.text`）の最初の1トークン目を出力する**前**に、サーバー内部で多段階の思考プロセス（Chain-of-Thought）を自己回帰的に生成するフェーズである。ここで生成された推論トークン数はレスポンスの `usage_metadata.thoughts_token_count` に記録される。重要な仕様として、**Thinking フェーズが進行している間は Streaming モードであってもクライアントへ可視テキストチャンクが一切届かない**ため、内部推論に要した時間（実測で約 `+9.0 ms / token`）はすべて初回応答待機時間（TTFT）へ直列に上乗せされる。
3. **Phase 3: Decoding（可視出力トークンの自己回帰生成：`output_token_count`）** ： Prefill と Thinking が完了した瞬間から、最終回答テキスト（`candidates_token_count` / `output_token_count`）を先頭から順に出力するフェーズである。

2. TTFT（Time to First Token）と Total 完了時間、および 2D トラジェクトリの重要性

上記の3フェーズを踏まえると、クライアント側で観測される2つの主要レイテンシ指標は次のように定式化される。

```
TTFT_streaming = T_Network_RTT + T_Prefill + T_Thinking
Total = TTFT_blocking = T_Network_RTT + T_Prefill + T_Thinking + T_Decoding
```

- **TTFT（Time to First Token / `time_to_first_token_ms`）**：リクエスト送信時刻（`t = 0 ms`）から、最初の非空テキストチャンクがクライアントに到着するまでの経過時間である。対話型 UI においてユーザーが「AI が反応した」と知覚する**体感待ち時間**そのものを表す。Blocking 呼び出しでは全文生成が終わるまでレスポンスが返らないため、常に `TTFT = Total` となる。
- **Total Completion Time（全体完了時間 / `stream_completed_ms`）**：リクエスト送信（`t = 0 ms`）から最終チャンク（EOF）の受信が完了するまでの合計時間である。
- **なぜ「2次元トラジェクトリ（経過時間 × 累積出力文字数）」が必要か**： 図2-1下段に示した実測比較（`thinking=0`）を見ると、完了時間（`Total`）だけを比較した場合は Blocking（`3,360 ms`）と Streaming（`3,192 ms`）は同水準に見える。しかし横軸に経過時間（`t_ms`）、縦軸にその時点で画面に表示された累積文字数（`chars`）をとった **2次元トラジェクトリ** を描くと、UX の決定的な違いが浮かび上がる。
  - **Blocking（図2-1左）**：`0 ms` から `3,360 ms` まで縦軸 `y = 0 chars`（画面白紙）のまま推移し、`3,360 ms` 時点で338文字が垂直ステップ状に一括出現する。
  - **Streaming（図2-1右）**：薄紫帯の Prefill 待機を終えた `1,205 ms`（**Blocking より `2,155 ms` / `64%` 早い時点**）で最初のチャンクが立ち上がり、薄青帯の区間（`1,205 → 3,192 ms`）にわたって滑らかな右肩上がりの傾斜で373文字が逐次表示される。人間がテキストを読む速度（日本語で約400〜600文字/分＝1秒あたり7〜10文字）よりも Gemini 3.8 Flash のデコード速度（1秒あたり約180〜200文字）の方が圧倒的に速いため、`1,205 ms` 以降の生成時間はユーザーの読解時間の裏に完全に隠蔽される。さらに第4章で扱う「段階1の受付文表示＋段階2の並列推論連結」のようなマルチステージ挙動も、2次元トラジェクトリを用いることで「初動の立ち上がり → 読解中のプラトー → 本回答の急勾配ストリーム」として視覚的に厳密評価できる。

2.2 Blocking vs. Streaming API の実装方法（Google AI Studio `google-genai` SDK）

本検証プロジェクト（`src/gemini_evaluation/llm_call.py`）では、Google AI Studio の公式 Python SDK である `google-genai` を用い、環境変数 `GEMINI_KEY` によるシンプルな API キー認証（`genai.Client(api_key=GEMINI_KEY)`）と非同期クライアント（`client.aio`）で全計測を統一している。

以下に、`gemini-3.8-flash` に対する `ThinkingConfig` **全6段階の解決関数**（`resolve_thinking_config`）と、**Blocking**（`generate_content`）および **Streaming**（`generate_content_stream`）の標準呼び出し実装を示す（※`gemini-3.8-flash` 固有の `thinking_level` / `thinking_budget` の仕様詳細および他モデルとの違いは **2.5節** で詳述する）。

```python
from collections.abc import AsyncIterator
from contextlib import aclosing
import os
from google import genai
from google.genai import types

# 1. Google AI Studio クライアントの初期化 (.env または環境変数の GEMINI_KEY を使用)
GEMINI_KEY = os.environ["GEMINI_KEY"]
MODEL = "gemini-3.8-flash"
client = genai.Client(api_key=GEMINI_KEY)


def resolve_thinking_config(thinking: str | int | None = 0) -> types.ThinkingConfig | None:
    """6段階の thinking 指定 (0, minimal, low, medium, high, default) を ThinkingConfig へ変換する。
    ※ gemini-3.8-flash では ThinkingLevel.MINIMAL が非対応 (400エラー) かつ ThinkingLevel.LOW が
      思考トークン 0 となるため、ベンチマーク上の 0 / minimal / low には後方互換の thinking_budget を割り当てて段階計測している(2.5節参照)。
    """
    if thinking is None or thinking == "default":
        return None  # サーバー既定の動的推論 (gemini-3.8-flash の既定は medium)
    if isinstance(thinking, int):
        return types.ThinkingConfig(thinking_budget=thinking)
    key = str(thinking).strip().lower()
    if key == "0":
        return types.ThinkingConfig(thinking_budget=0)
    if key == "minimal":
        return types.ThinkingConfig(thinking_budget=256)
    if key == "low":
        return types.ThinkingConfig(thinking_budget=4096)
    level_map = {
        "medium": types.ThinkingLevel.MEDIUM,
        "high": types.ThinkingLevel.HIGH,
    }
    return types.ThinkingConfig(thinking_level=level_map[key])


# 2. Blocking 呼び出し (全文生成完了まで待機: TTFT == Total)
async def generate_text_blocking(prompt: str, thinking: str | int | None = 0) -> str:
    cfg = resolve_thinking_config(thinking)
    resp = await client.aio.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(thinking_config=cfg) if cfg else None,
    )
    return resp.text or ""


# 3. Streaming 呼び出し (TTFT 到達直後からチャンクを逐次 yield)
async def stream_text_progressive(prompt: str, thinking: str | int | None = 0) ->
AsyncIterator[str]:
    cfg = resolve_thinking_config(thinking)
    stream = await client.aio.models.generate_content_stream(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(thinking_config=cfg) if cfg else None,
    )
    async with aclosing(stream):
```

```
        async for chunk in stream:
            if chunk.text:
                yield chunk.text
```

2.3 Pydantic 構造化出力と不完全 JSON 逐次パース（Partial JSON Streaming）の仕組み

対話システムにおいて「質問の難易度に応じて内部推論（Thinking）の ON/OFF を自動判定したい」場合や、「画面表示テキストと同時に UI 制御用のフラグ・カテゴリを構造化取得したい」場合には、`GenerateContentConfig` に `response_mime_type="application/json"` と Pydantic の `BaseModel` クラス（`response_schema=...`）を指定する **構造化出力（Structured Outputs）** を利用する。

しかし、構造化出力をストリーミング（`generate_content_stream`）と組み合わせる際には、サーバー側の **FSM（有限状態機械）文法コンパイル特性** と、クライアント側の **不完全 JSON 逐次パース手法** の2点を正しく設計する必要がある。図2-2に、実機で計測した Cold vs. Warm スキーマコンパイル実験（`load_schema_experiment()`、全8試行）と、`pydantic_core.from_json(..., allow_partial="trailing-strings")` による逐次差分抽出パイプラインを示す。

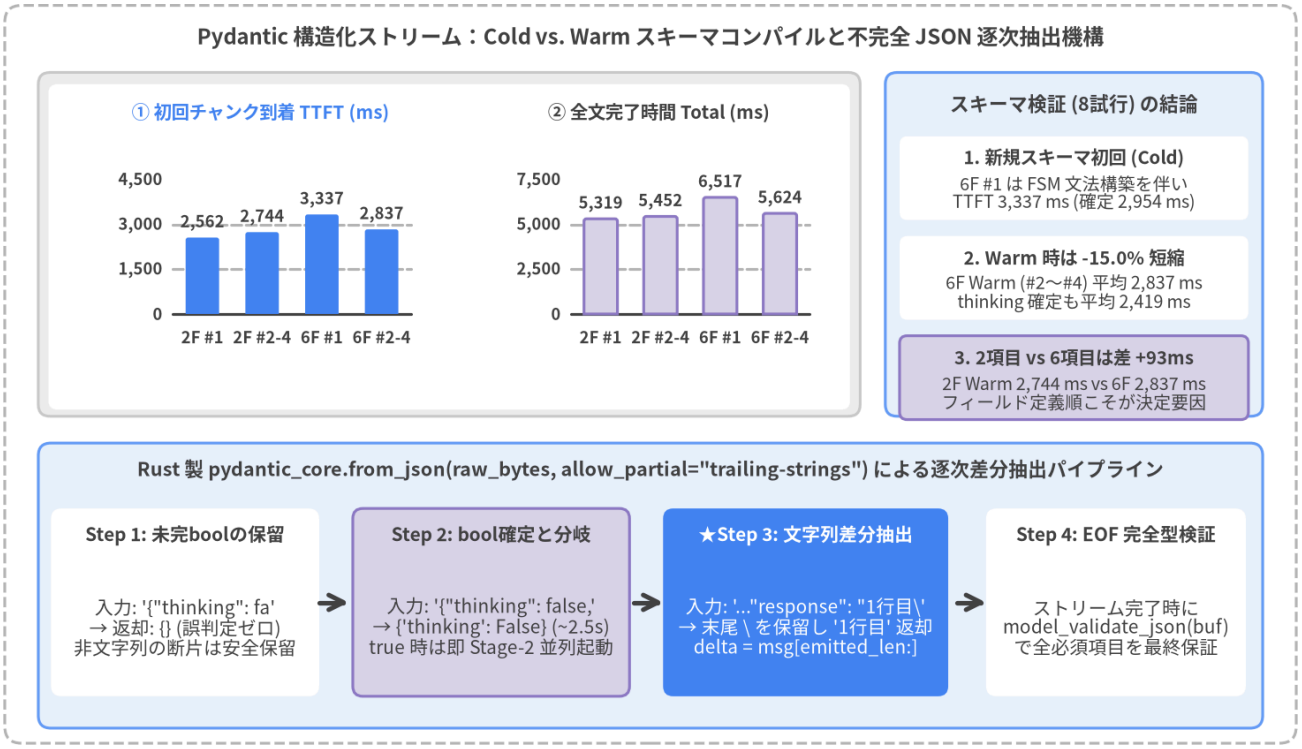


図2-2: Pydantic response_schema の Cold vs. Warm コンパイル実験（2項目 vs. 6項目、全8試行）と pydantic_core.from_json による逐次差分抽出パイプライン

---

1. サーバー側 Constrained Decoding（制約付きデコード）と Cold vs. Warm 文法コンパイル

`response_mime_type="application/json"` と `response_schema=AdaptiveStreamResponse` を指定すると、Google AI Studio バックエンドは与えられた Pydantic モデルの JSON Schema を解析し、デコーダ層で構文違反トークンの生成確率をマスクする **有限状態機械（FSM: Finite-State Machine）文法** へコンパイルする。これにより、マークダウンのコードフェンス混入や閉じ括弧漏れが数学的にゼロとなる。

図2-2上段および下表は、最小2フィールドの `AdaptiveStreamResponse(thinking: bool, response: str)`（`minimal_2field`）と、`Literal` 列挙型2項目・真偽値・文字列リストを加えた拡張6フィールドの `RichStreamResponse`（`rich_6field`）に対して、同一プロンプトを4回連続実行した実測値（`src/gemini_evaluation/adaptive_thinking_log.json`）である。なお、すべての試行において先頭フィールド `thinking` の確定時刻（`flag_decided_ms`）は第2フィールド `response` の初回チャンク表示時刻（`ttft_ms`）以前に成立（`flag_decided_ms <= ttft_ms < total_ms`）している。

スキーマ構成 (`schema_label`)	試行	キャッシュ状態	`thinking` 確定 (`flag_decided_ms`)	初回チャンク (`ttft_ms`)	全体完了 (`total_ms`)	出力文字数
`minimal_2field` (`AdaptiveStreamResponse`)	#1	Warm (本実験直後)	2,562.2 ms	2,562.2 ms	5,319.0 ms	496 字
`minimal_2field` (`AdaptiveStreamResponse`)	#2	Warm	2,294.1 ms	2,294.1 ms	4,551.2 ms	455 字
`minimal_2field` (`AdaptiveStreamResponse`)	#3	Warm	2,194.2 ms	2,492.2 ms	4,950.2 ms	579 字
`minimal_2field` (`AdaptiveStreamResponse`)	#4	Warm	3,446.8 ms	3,446.8 ms	6,855.2 ms	686 字
`rich_6field` (`RichStreamResponse`)	#1	Cold (新規初回)	2,954.3 ms	3,337.3 ms	6,517.4 ms	507 字
`rich_6field` (`RichStreamResponse`)	#2	Warm	2,157.1 ms	2,504.4 ms	4,526.9 ms	360 字
`rich_6field` (`RichStreamResponse`)	#3	Warm	2,780.6 ms	3,259.3 ms	5,838.9 ms	422 字
`rich_6field` (`RichStreamResponse`)	#4	Warm	2,320.5 ms	2,747.5 ms	6,504.9 ms	679 字

- **新規スキーマ初回（Cold `#1`）と Warm `#2〜#4` の比較（`-15.0%` 短縮）**：`minimal_2field`（`AdaptiveStreamResponse`）は直前の本実験（45試行）で既にコンパイル済みだったため `#1` から `2,562.2 ms`（`#2〜#4` 平均 `2,744.4 ms`、全4回平均 `2,698.8 ms`）で応答した。一方、新規の6フィールド構成 `rich_6field` を初めて送信した `rich_6field #1`**（Cold）では、FSM 文法コンパイルに伴い** `TTFT = 3,337.3 ms`（`flag_decided_ms = 2,954.3 ms`）を要したが、直後の `#2〜#4`**（Warm）ではコンパイル済みキャッシュが適用され、平均 TTFT は** `2,837.1 ms`（`2,504.4〜3,259.3 ms`、**Cold 比** `-500.2 ms` / `-15.0%` **短縮）**、`thinking` フラグ確定時刻も平均 `2,419.4 ms` へ短縮された。
- **フィールド数（2項目 vs. 6項目）よりも「フィールド定義順」が TTFT を支配する**：Warm `#2〜#4` 同士を比較すると、2フィールド構成（平均 `2,744.4 ms`）と6フィールド構成（平均 `2,837.1 ms`）の TTFT 差はわずか `+92.7 ms`（`+3.4%`）に過ぎない。これは `RichStreamResponse` において `thinking: bool` **を先頭（第1フィールド）に置き、短い固定列挙型（**`category`, `urgency`, `needs_human_escalation`**）の直後に** `response: str` **を配置し、可変長リスト** `suggested_followups: list[str]` **を** `response` **の後ろ（末尾）に配置したため** である。JSON ストリーミングは定義順にトークンを生成するため、ルーティング用フラグを先頭、画面表示用の `response` を可変長配列の前に置くことが鉄則となる。

2. `pydantic_core.from_json(..., allow_partial="trailing-strings")` による逐次差分抽出

`generate_content_stream` で構造化 JSON を受信する際、ストリーム途中のチャンクには閉じ括弧のない JSON 断片（例：`'{"thinking": false, "response": "いつもご利用'`）が入るため、

SDK 標準の `chunk.parsed` はストリーム完了まで常に `None` となる。

そこで `src/gemini_evaluation/adaptive_thinking.py` では、Pydantic v2 に標準搭載されている Rust 製パーサー `pydantic_core.from_json(raw_bytes, allow_partial="trailing-strings")` を用いてチャンク到着ごとに累積バッファを解析している。この方式は以下の決定的な利点を持つ。

1. **未完プリミティブの安全な保留**：`'{"thinking": fa'` のように真偽値リテラルが途中で切れている段階では `{}` を返し、`false` または `true` が完結した瞬間に初めて `{'thinking': False}` を返すため、不完全なトークンによる誤分岐が起きない。
2. **エスケープシーケンス分断への耐性と `O(1)` 差分スライス**：`"response"` 文字列の末尾がバックスラッシュ `\`（例：`\n` の1文字目）で分断された場合も、未完の `\` を内部保留して確定済み文字のみを返す。そのため、既にクライアントへ送出した文字数 `emitted_resp_len` を記録しておき、`delta = resp_so_far[emitted_resp_len:]` をスライスするだけで、文字化けや JSON 構文漏出を一切起こさずにプレーンテキスト同等のストリーミング表示を実現できる。

```python
from typing import Any
from pydantic import BaseModel
from pydantic_core import from_json


class AdaptiveStreamResponse(BaseModel):
    thinking: bool  # 第1フィールド：初回チャンク以前に確定するルーティングフラグ
    response: str   # 第2フィールド：逐次ストリーム表示する回答本文（または受付1文）


def parse_partial_json_dict(raw_buf: str) -> dict[str, Any] | None:
    """不完全なストリーミング JSON 文字列を Rust 製 pydantic_core で安全に辞書へ変換する。"""
    if not raw_buf:
        return None
    try:
        parsed = from_json(raw_buf.encode("utf-8"), allow_partial="trailing-strings")
        return parsed if isinstance(parsed, dict) else None
    except ValueError:
        return None
```

2.4 計測基盤アーキテクチャ（`ExecutionTracer`）

本レポートの全ベンチマーク（`src/gemini_evaluation/benchmark_thinking.py`、`benchmark_adaptive_thinking.py`、`benchmark_multiturn_chat.py`）は、

`src/gemini_evaluation/tracer.py` に実装された共通計測クラス `ExecutionTracer` を用いて計測・シリアライズされている。

`ExecutionTracer` が記録するマイクロ秒精度マイルストーンとトークン内訳

`ExecutionTracer` は、OS のシステム時計補正の影響を受けない単調増加タイマー `time.perf_counter()` を基準時計として使用し、各 API 呼び出しで以下の時系列イベントとメタデータを記録する。

記録カテゴリ	フィールド名 (`ExecutionTracer`)	計測タイミング・定義
リクエスト開始	`start` (`t_ms = 0.0`, `chars = 0`)	`tracer.start_gemini()` 呼び出し時点（API 呼び出し直前）の `time.perf_counter()` 原点。
推論・初動待機終端	`thinking_end` (`t_ms = ttft_ms`, `chars = 0`)	最初の非空チャンク到着直前に `(ttft_ms, 0)` を記録し、2D トラジェクトリ上の水平待機プラトー（Prefill + Thinking）を表現。
ストリーミングチャンク	`chunk_1 .. chunk_N` (`t_ms`, `chars`)	`tracer.record_chunk(delta)` が呼ばれるたびに経過ミリ秒 `t_ms` と累積文字数 `_cum_chars` を追記（Blocking 時は `(total_ms, 0)` と `(total_ms, len(text))` のステップを記録）。
適応型ルーティング指標	`flag_decided_ms` / `stage2_first_chunk_ms`	Pattern A / B / C において `thinking` フラグが確定した時刻と、並列起動された Stage-2（`thinking="low"`）の初回チャンクが到着した時刻。
トークン使用量内訳	`prompt_token_count` / `cached_content_token_count` / `thoughts_token_count` / `output_token_count`	`usage_metadata` から抽出した入力トークン数、暗黙キャッシュヒットトークン数、内部推論トークン数、および可視出力トークン数。

すべての計測結果は人間が読みやすい YAML（`*_log.yaml`）およびプログラム処理用の JSON / JSONL（`*_log.json`、`*_runs.jsonl`、`*_turns.jsonl`）として保存され、本レポート内のすべての Drawlib チャートは `examples/doc/gemini_latency/utils.py` のローダー関数を介してこれらの実測ログから直接描画されている。

2.5 Gemini 3.5 / 3.8 / 4 シリーズのモデル分類と API オプション仕様の網羅検証

本レポートの第3章〜第5章では `gemini-3.8-flash` をベンチマーク対象モデルとして使用しているが、Google AI Studio（Gemini API）では **Gemini 3.5 / 3.8 シリーズ** から最新の **Gemini 4 シリーズ（`Hydrogen` / `Argon` / `Carbon`）** にかけて、`ThinkingConfig`（`thinking_level` と `thinking_budget` の扱い）や **Context Caching（Explicit / Implicit キャッシュとマルチターン末尾再処理ウィンドウ）**、サンプリングパラメータの仕様が大きく変化している。

本節では、公式ドキュメントの仕様と実機 API（`client.models.get()` および `generate_content()`）による検証結果を突き合わせ、**現行モデルの選び分け**と**モデル別の利用可能オプション・実測挙動**を3つの詳細な一覧表（表2-5A〜表2-5C）に整理する。

1. 現行モデルの分類・ポジショニングと選び分け指針（表2-5A）

現在（2026年11月時点）、旧世代の `gemini-2.5-flash`（`gemini-3.5-flash-lite` へ自動転送）、`gemini-2.5-pro`（`gemini-3.8-flash` へ自動転送）、および `gemini-3.5-flash`（`gemini-3.6-flash` へ自動転送）はすべて Deprecated（非推奨）となっている。一方で **Gemini 4 世代には「Lite」相当の超軽量モデルがまだ存在しない**ため、音声（Live / TTS）や画像生成（Nano Banana 系）の専用モデルを除けば、テキスト・マルチモーダル生成は実質的に下表の **現行 5 モデル（`3.5-flash-lite`、`3.8-flash`、`4-hydrogen`、`4-argon`、`4-carbon`）** から選択すれば全領域をカバーできる。

モデルID（`model`）	提供状態・世代	価格 (Input / Output 1M tok)	コンテキスト枠 / 単発出力上限	既定 `thinking_level`	ポジショニングと実務上の選択基準
`gemini-3.5-flash`	Deprecated *(3.6-flash へ自動転送)*	$1.50 / $7.50 *(3.6-flash 価格)*	入力 `1,048,576` 出力 `65,536` (個別枠)	`medium`	**【旧世代 Flash（非推奨）】** ・現在はリクエストが自動的に `gemini-3.6-flash` へルーティングされる。 ・新規開発では選ぶ必要はなく、`gemini-4-hydrogen` または `gemini-3.8-flash` へ移行する。
`gemini-3.5-flash-lite`	**現役 Stable** *(3.x 最新 Lite)*	**$0.30 / $2.50**	入力 `1,048,576` 出力 `65,536` (個別枠)	`minimal` *(思考 0 tok)*	**【最速・最安の軽量ワークホース（Lite 枠）】** ・Gemini 4 世代に Lite が存在しないため、**新規開発の軽量・高スループット枠における公式推奨モデル**（`2.5-flash` の公式移行先）。 ・既定の `minimal` で思考トークン `0` の最速応答を返しつつ、サブエージェントの自律計画時のみ `medium` / `high` で思考させる使い分けに最適。
`gemini-3.8-flash`	**現役 Stable** *(本レポート検証対象)*	**$1.50 / $7.50**	入力 `1,048,576` 出力 `65,536` (個別枠)	`medium`	**【3.x 世代の最終完成版 Flash】** ・`2.5-pro` の公式移行先。基本は同価格の後継 `gemini-4-hydrogen` への移行が推奨される。 ・ただし ① `thinking_level="low"`（**または** `thinking_budget=0`）**で思考トークンを完全に** `0`（**低遅延**）**にしたい場合**、および ② **Explicit Context Caching（**`caches.create`**）を利用したい場合** は `4-hydrogen` が非対応のため、あえて `3.8-flash` を選択する明確なメリットがある。
`gemini-4-hydrogen`	**現役 Stable** *(Flash 後継推奨)*	**$1.50 / $7.50** *(Free / Paid 両対応)*	**統合 1M 枠**（`1,048,576`）単発出力 `65,536` *(Long Decode 最大 1M)*	`high`	**【Gemini 4 標準メインストリーム】** ・`3.8-flash` と同価格で推論・コーディング能力が向上した現行の基本推奨モデル。 ・常時思考型（思考 0 トークン化は不可）であり、Hydrogen 専用の最高推論レベル `xhigh` と **Long Decode（**`automatic_continuation=True` **による最大 1M トークン連続出力）** に対応する。

モデルID (`model`)	提供状態・世代	価格 (Input / Output 1M tok)	コンテキスト枠 / 単発出力上限	既定 `thinking_level`	ポジショニングと実務上の選択基準
`gemini-4-argon`	**現役 Stable** *(Paid Tier 限定)*	**$2.00 / $10.00** *(2026/12/1 以降 $4 / $20)*	**統合 1M 枠** (`1,048,576`) REST 単発 `32,768` / Stream `65,536` *(Long Decode 最大 1M)*	`medium`	**【高効率フロンティア推論（Pro 相当枠）】** ・`hydrogen` では解けない高度なソフトウェア設計・数学・複雑なマルチステップ推論向け。 ・`carbon` に迫る推論精度を維持しつつ、1ターンあたりの思考トークン消費量と応答レイテンシを抑えたバランス型上位モデル。
`gemini-4-carbon`	**現役 Stable** *(Paid Tier 限定)*	**$2.00 / $10.00** *(2026/12/1 以降 $4 / $20)*	**統合 1M 枠** (`1,048,576`) REST 単発 `32,768` / Stream `65,536` *(Long Decode 最大 1M)*	`medium`	**【最高峰フロンティア推論（最難関タスク枠）】** ・1ターンあたり最も多くの思考トークンを投入し、難解なバグ解析・科学推論・サイバーセキュリティ防御など、レイテンシよりも**回答の到達精度が最優先**される場面の最終選択肢。

---

2. モデル別 `ThinkingConfig`（`thinking_level` vs. `thinking_budget`）仕様と実機検証マトリクス（表2-5B）

Gemini API における内部推論（Thinking）の制御パラメータには、Gemini 2.5 世代で導入された数値上限指定の `thinking_budget` と、Gemini 3 以降で導入された抽象度指定の `thinking_level` の2種類が存在する。

ここで最も重要な設計原則として、`thinking_budget` **は Gemini 3 以降の全モデルにおいて公式に非推奨（Deprecated）** であり、**「思考をオフ（0トークン化）または最小化したい場合は `thinking_budget=0` ではなく、各モデルがサポートする最小の `thinking_level`（`minimal` または `low`）を指定する」** のが正規の仕様である（なお、同一リクエスト内で `thinking_level` と `thinking_budget` を同時指定すると全モデルで `400` エラーとなる）。

下表（表2-5B）に、各モデルに対して `thinking_level` 全5種類（`minimal`, `low`, `medium`, `high`, `xhigh`）および後方互換の `thinking_budget`（`0`, `256`, `4096`）を送信した際の実機 API 挙動と生成された思考トークン数（`usage_metadata.thoughts_token_count`：単純な1問1答プロンプト／複雑な5人座席配置パズルプロンプト）を示す。

指定パラメータ (`ThinkingConfig`)	`gemini-3.5-flash` ***(3.6-flash へ自動転送)***	`gemini-3.5-flash-lite` ***(現役 Lite)***	`gemini-3.8-flash` ***(本レポート検証対象)***	`gemini-4-hydrogen` ***(4 標準モデル)***	`gemini-4-argon` / `gemini-4-carbon`
サポートされる `thinking_level`	`minimal`, `low`, `medium`, `high` (4段階)	`minimal`, `low`, `medium`, `high` (4段階)	`low`, `medium`, `high` (3段階)	`low`, `medium`, `high`, `xhigh` (4段階)	`low`, `medium`, `high` (3段階)

指定パラメータ（`ThinkingConfig`）	`gemini-3.5-flash` *(3.6-flash へ自動転送)*	`gemini-3.5-flash-lite` *(現役 Lite)*	`gemini-3.8-flash` *(本レポート検証対象)*	`gemini-4-hydrogen` *(4 標準モデル)*	`gemini-4-argon` / `gemini-4-carbon`
未指定（`None` / デフォルト）	既定 = `medium` 単純: `69 tok` 複雑: `2,711 tok`	既定 = `minimal` 単純: `0 tok`（`None`） 複雑: `0 tok`（`None`）	既定 = `medium` 単純: `0 tok`（`None`） 複雑: `1,303 tok`	既定 = `high` 単純: `68 tok` 複雑: `1,286 tok`	既定 = `medium` 複雑推論で動的思考
`thinking_level="minimal"`	✅ **対応（思考オフ相当）** 単純: `0 tok` 複雑: `0 tok`	✅ **対応（既定・思考オフ）** 単純: `0 tok` 複雑: `0 tok`	❌ `400 INVALID_ARGUMENT` *(MINIMAL is not supported)*	❌ `400 INVALID_ARGUMENT` *(MINIMAL is not supported)*	❌ `400 INVALID_ARGUMENT` *(MINIMAL is not supported)*
`thinking_level="low"`	✅ **対応（軽量〜中程度思考）** 単純: `70 tok` 複雑: `2,051 tok`	✅ **対応（極小思考）** 単純: `41 tok` 複雑: `0 tok`	✅ **対応（実質思考オフ）** 単純: `0 tok` 複雑: `0 tok`	✅ **対応（最小レベル・常時思考）** 単純: `35 tok` 複雑: `1,324 tok`	✅ **対応（最小レベル）** 単純: `0 tok` 複雑: 軽量思考
`thinking_level="medium"`	✅ **対応（既定）** 単純: `73 tok` 複雑: `2,613 tok`	✅ **対応（本格思考開始）** 単純: `53 tok` 複雑: `4,175 tok`	✅ **対応（既定）** 単純: `0 tok` 複雑: `1,403 tok`	✅ **対応** 単純: `36 tok` 複雑: `1,528 tok`	✅ **対応（既定）**
`thinking_level="high"`	✅ **対応** 単純: `70 tok` 複雑: `3,921 tok`	✅ **対応** 単純: `58 tok` 複雑: `4,224 tok`	✅ **対応** 単純: `61 tok` 複雑: `1,636 tok`	✅ **対応（既定）** 単純: `67 tok` 複雑: `1,217 tok`	✅ **対応**
`thinking_level="xhigh"`	❌ `400 INVALID_ARGUMENT` *(XHIGH is not supported)*	❌ `400 INVALID_ARGUMENT` *(XHIGH is not supported)*	❌ `400 INVALID_ARGUMENT` *(XHIGH is not supported)*	✅ **対応（Hydrogen 専用）** 単純: `84 tok` 複雑: `3,376 tok`	❌ `400 INVALID_ARGUMENT` *(XHIGH is not supported)*
`thinking_budget=0` *(非推奨・レガシー指定)*	⚠️ 後方互換で動作 単純: `0 tok` / 複雑: `0 tok`	❌ `400 INVALID_ARGUMENT` *(0 指定はエラーで拒否)*	⚠️ 後方互換で動作 単純: `0 tok` / 複雑: `0 tok` *(本検証の `"0"` で使用)*	⚠️ `low` **に強制変換（警告）** 単純: `32 tok` / 複雑: `1,398 tok` *(思考 0 にはならない)*	⚠️ 最寄りの `thinking_level` に自動変換（思考オフ不可）
`thinking_budget=256` *(非推奨・レガシー指定)*	⚠️ 上限256を超えて思考 単純: `63 tok` / 複雑: `1,486 tok`	⚠️ 上限256を超えて思考 単純: `50 tok` / 複雑: `2,009 tok`	⚠️ 後方互換で動作 単純: `0 tok` / 複雑: `0 tok` *(本検証の `"minimal"` で使用)*	⚠️ `thinking_level` に丸め込み 単純: `69 tok` / 複雑: `1,346 tok`	⚠️ 最寄りの `thinking_level` に自動変換
`thinking_budget=4096` *(非推奨・レガシー指定)*	⚠️ `medium`〜`high` 相当 単純: `68 tok` / 複雑: `3,355 tok`	⚠️ `medium` 相当 単純: `62 tok` / 複雑: `3,449 tok`	⚠️ **中間思考量を生成** 単純: `0 tok` / 複雑: `351〜673 tok` *(本検証の `"low"` で使用)*	⚠️ `thinking_level` に丸め込み 複雑: `1,471 tok`	⚠️ 最寄りの `thinking_level` に自動変換

表2-5B から導かれる重要な実装上の注意点

1. **モデルによって「思考オフ（`0 tok`）」を実現する正規の指定方法が異なる：**
   - `gemini-3.5-flash-lite` / `gemini-3.5-flash`（`3.6-flash`）：`thinking_level="minimal"` を指定する（特に `3.5-flash-lite` はデフォルトが

`"minimal"` のため未指定でも `0 tok` となる一方、`thinking_budget=0` **を渡すと** `400 INVALID_ARGUMENT` **でクラッシュする**ため注意が必要である）。

- `gemini-3.8-flash`：`"minimal"` が廃止（`400` エラー）された代わりに、`thinking_level="low"` **が実質的な思考オフ（実測** `0 tok`**）として機能**する。
- `gemini-4-hydrogen`：公式仕様として「思考の無効化は不可（cannot disable thinking）」と定められており、`thinking_level="low"` や `thinking_budget=0` を渡しても必ず思考トークン（複雑問題で約 `1,300〜1,400 tok`）が生成される。

2. **本レポートの検証コード（**`src/gemini_evaluation/llm_call.py`**）における 6 段階ラベルの定義理由**：
   - 表2-5B の `gemini-3.8-flash` 列から分かるとおり、`gemini-3.8-flash` の公式 `thinking_level` は `low`（実測 `0 tok`）の次がいきなり `medium`（実測 `1,300〜1,400 tok`）へ跳躍する。
   - そこで第3章〜第4章のベンチマークでは、「思考ゼロ（`0 tok`）」と「深い思考（`1,300+ tok`）」の間にある **中程度の思考トークン領域（**`350〜670 tok` **帯）でのレイテンシと計算精度の推移**を精密に測定するため、`resolve_thinking_config()` において `"0"` → `thinking_budget=0`（実測 `0 tok`）、`"minimal"` → `thinking_budget=256`（実測 `0 tok`）、`"low"` → `thinking_budget=4096`（実測 `351〜673 tok`）、`"medium"` → `ThinkingLevel.MEDIUM`、`"high"` → `ThinkingLevel.HIGH`、`"default"` → `None`（サーバー既定＝`medium`）というマッピングを採用している。

---

3. Context Caching（暗黙・明示キャッシュ）および API 制約の世代間比較（表2-5C）

長大なシステムプロンプトやマルチターン対話履歴の Prefill コスト・TTFT を削減する **Context Caching** や、サンプリング系パラメータの扱いについても、Gemini 3.5 / 3.8 / 4 で下表（表2-5C）の仕様差が存在する。

機能・パラメータ項目	`gemini-3.5-flash` / `gemini-3.5-flash-lite`	`gemini-3.8-flash` ***(本レポート検証対象)***	`gemini-4-hydrogen` / `argon` / `carbon`
Implicit Context Caching *(暗黙的キャッシュ：自動適用)*	✅ **デフォルト ON**（設定不要） ・公称最小入力トークン数：`4,096 tok` ・キャッシュヒット分は Prefill 計算をスキップし入力単価を大幅割引	✅ **デフォルト ON**（設定不要） ・公称最小入力トークン数：`4,096 tok` ・**実測では総入力が** `2,048 tok` **を超えた直後の Turn 8（**`2,686 tok`**）からキャッシュヒット開始**（第5章参照）	✅ **デフォルト ON**（設定不要・**95% 割引**） ・公称最小入力トークン数：`4,096 tok` ・**マルチターン時の「末尾 2,050 トークン再処理ルール」が公式ドキュメントに明文化**

機能・パラメータ項目	`gemini-3.5-flash` / `gemini-3.5-flash-lite`	`gemini-3.8-flash` ***(本レポート検証対象)***	`gemini-4-hydrogen` / `argon` / `carbon`
マルチターン対話におけるキャッシュ境界の内部アルゴリズム *(なぜ約2,050 tok から始まり末尾が非キャッシュで残るのか)*	プレフィックス一致に対してキャッシュが自動構築されるが、直近ターンの末尾ウィンドウは通常レートで再処理される。	第5章の50ターン連続対話ベンチマークにて、**Turn 8（`2,686 prompt_tokens`）で `703 cached_tokens` が初発動し、以降 Turn 50 まで常に `Prompt - Cached ≈ 2,000 tokens`（末尾約2,000トークン）が非キャッシュとして残る**挙動を実測確認。	Gemini 4 公式ドキュメントにて以下の内部仕様が正式に明記された：*"新規リクエストの増分（モデル出力・思考・関数結果を含む）が **2,050 トークン以上** の場合のみ前回リクエスト全体がキャッシュされ、**2,050 トークン未満の増分では前回リクエストの末尾 2,050 トークンを通常入力レートで再処理する**"*
Explicit Context Caching（`client.caches.create` / `createCachedContent`）	✅ **サポート**（`v1beta`） ・任意の TTL を指定して明示的にキャッシュオブジェクトを作成・再利用可能	✅ **サポート**（`v1beta`） ・`models.get` の `supported_actions` に `createCachedContent` を含む	❌ **廃止（全モデル非サポート・エラー）** ・`createCachedContent` は削除され、Implicit Caching のみに完全一本化
サンプリングパラメータ（`temperature`, `top_p`, `top_k`, `candidate_count`）	⚠️ **Deprecated（非推奨・API側で無視）** ・`candidate_count` は非対応	⚠️ **Deprecated（非推奨・API側で無視）** ・推論品質維持のためデフォルト値固定（リクエストから削除推奨）	⚠️ **Deprecated（非推奨・API側で無視）** ・有効範囲内の値は無視され、範囲外の値を渡すと `400` バリデーションエラー
Model ロールでのプリフィル終了（`contents` 末尾が `role="model"`）	❌ `400 INVALID_ARGUMENT` **エラー** *(3.6-flash / 3.5-flash-lite 以降で禁止)*	❌ `400 INVALID_ARGUMENT` **エラー** ・出力形式の強制には `system_instruction` または `response_schema` を使用する	❌ `400 INVALID_ARGUMENT` **エラー**

表2-5C から導かれるコンテキストキャッシュの重要インサイト

公式ドキュメントの一覧表では、Gemini 3.8 や Gemini 4 の Implicit Context Caching の最小入力トークン数は `4,096 tokens` と記載されている。しかし第5章の実機50ターン検証（`gemini-3.8-flash`）では、`4,096 tokens` 未満である **Turn 8（`prompt_token_count = 2,686`）の時点で既に `cached_content_token_count = 703` のキャッシュヒットが自動的に始まっている**。

この一見矛盾する現象は、Gemini 4 の公式リリースノートで明文化された **「1ターンあたりの増分が 2,050 トークン未満のマルチターン対話では、前回リクエストの末尾 2,050 トークンを通常レートで再処理し、それより前のプレフィックスをキャッシュする」** という内部ルールによって完全に説明できる。すなわち、総入力トークン数が **`2,048〜2,050 tokens` の末尾再処理ウィンドウを超えた瞬間（Turn 7 の `2,298 tok` → Turn 8 の `2,686 tok`）から、末尾約 `2,000 tokens` を除いた先頭プレフィックス部分（`2,686 - 2,000 ≈ 686〜703 tok`）がキャッシュにヒットし始める**ためである（詳細は第5章 5.2節で検証する）。

3. パラメータとレイテンシー

第2章では、`gemini-3.8-flash` に対するブロッキング呼び出し（`generate_content`）とストリーミング呼び出し（`generate_content_stream`）の基本構造、および Gemini 3.5 / 3.8 / 4 シリーズのモデル仕様を確認した。本章では、`src/gemini_evaluation/benchmark_thinking.py`、`src/gemini_evaluation/benchmark_adaptive_thinking.py`、および `src/gemini_evaluation/benchmark_cross_model_thinking.py` による実機計測データに基づき、レイテンシーを支配する最大要因である `ThinkingConfig`**（内部推論トークン制御）** の全6段階の挙動（3.1〜3.2節）、推論トークン数と初回トークン到着時間（TTFT）の線形スケーリング則（3.3節）、質問難易度（Tier 1〜3）に応じた「応答速度と計算正確性のトレードオフ」（3.4節）、そして **現行5モデル（`3.5-flash-lite` / `3.8-flash` / `4-hydrogen` / `4-argon` / `4-carbon`）× 各 `thinking_level` に同一の簡単クエリ5種・複雑クエリ5種を投入した170試行の横断レイテンシー比較（3.5節）** を定量的に解明する。

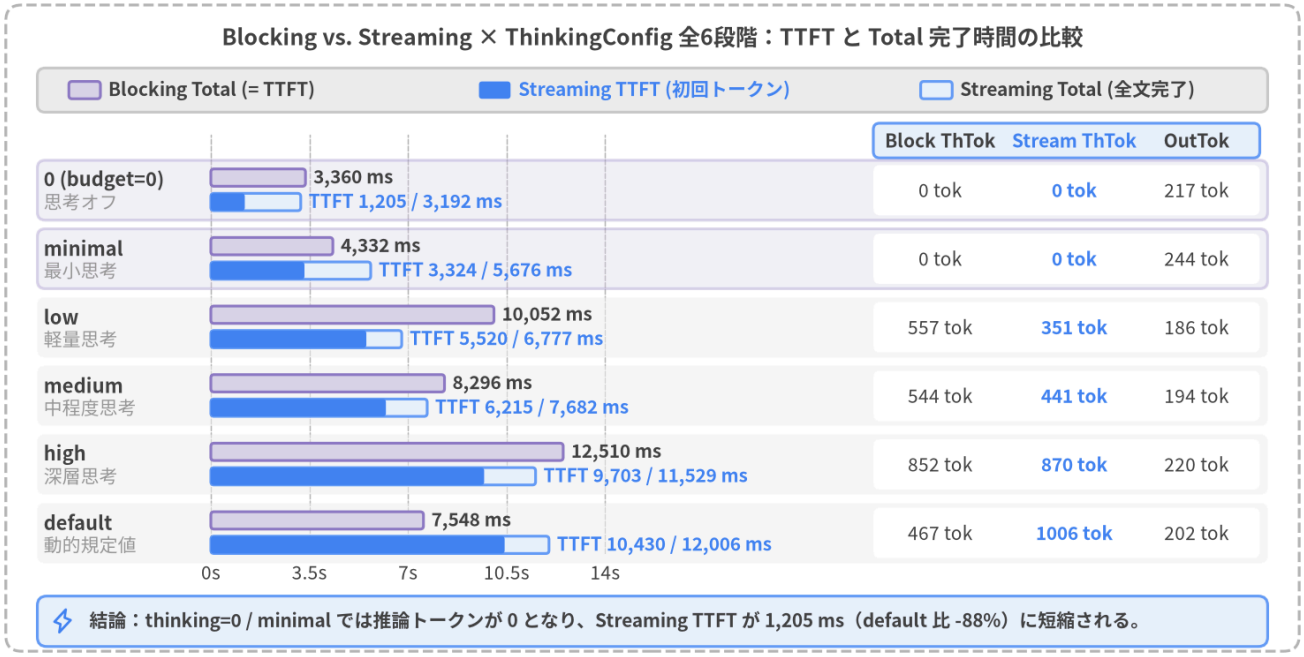


図3-1: Blocking (generate_content) と Streaming (generate_content_stream) × ThinkingConfig 全6段階における TTFT・完了時間（Total）・推論トークン数の比較

3.1 `ThinkingConfig` 全6段階の仕様と Blocking vs. Streaming 実測ベンチマーク

Google AI Studio（`google-genai` SDK）から `gemini-3.8-flash` を呼び出す際、モデルが最初の可視テキストチャンクを出力する前に内部で実行する推論プロセス（Thinking）は `types.ThinkingConfig` によって制御される。第2章 2.5節で検証したとおり、`gemini-3.8-flash` では `ThinkingLevel.MINIMAL` が非対応（`400` エラー）かつ `ThinkingLevel.LOW` が極小思考（定型質問では `0 tok`）となるため、評価実装（`src/gemini_evaluation/llm_call.py` の `resolve_thinking_config()`）では後方互換の `thinking_budget` と `ThinkingLevel` を組み合わせた以下の **全6段階のオプション** を定義し、推論トークン数に応じた段階的なレイテンシ推移を計測している（純粋な `thinking_level` のみを用いた5モデル横断比較は **3.5節** を参照）。

```
from google import genai
from google.genai import types

client = genai.Client(api_key=GEMINI_KEY)

# 例1: 内部推論を完全に無効化し、最短 TTFT を得る設定
config_zero = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_budget=0),
)

# 例2: 複雑な多段階計算向けに中程度の内部推論を有効化する設定
config_low = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_budget=4096),
)
```

オプション指定値（`thinking=`）	正規化ラベル	SDK 設定（`types.ThinkingConfig`）	実測推論トークン数（`ThTok`）	動作特性と推奨ユースケース
`0`（または `"0"`）	`"0"`	`ThinkingConfig(thinking_budget=0)`	`0 tok`	推論トークン上限を **0 tok** に固定。内部思考を完全にスキップし、最速の TTFT（約 `1,205〜1,546 ms`）を実現する。挨拶・定型案内・分類ルーターに最適。
`1`（または `"minimal"`）	`"minimal"`	`ThinkingConfig(thinking_budget=256)`	`0 tok`	極小バジェット指定。定型プロンプトでは実質 `0 tok` となり、推論トークンを消費しない高速応答を示す。
`2`（または `"low"`）	`"low"`	`ThinkingConfig(thinking_budget=4096)`	`351〜557 tok`	軽量〜中程度の内部推論を実行。TTFT は約 `4.3〜5.5 秒` となるが、多段階の算数・見積もり比較の計算ミスを防ぐのに十分な推論量を確保する。
`3`（または `"medium"`）	`"medium"`	`ThinkingConfig(thinking_level=MEDIUM)`	`441〜592 tok`	中程度の内部推論を実行。TTFT は約 `6.2〜6.7 秒` を要する。
`4`（または `"high"`）	`"high"`	`ThinkingConfig(thinking_level=HIGH)`	`817〜870 tok`	深い内部推論を実行。複雑な論理パズルや高度な推論に向くが、定型対話では `9.0〜9.7 秒` の待機が発生する。

オプション指定値（`thinking=`）	正規化ラベル	SDK 設定（`types.ThinkingConfig`）	実測推論トークン数（`ThTok`）	動作特性と推奨ユースケース
`None`（または `"default"`）	`"default"`	指定なし（動的デフォルト＝`MEDIUM`）	`467〜1,006 tok`	`thinking_config` 省略時のデフォルト挙動。プロンプトに応じて動的に推論トークンが割り当てられ、定型質問でも `7.5〜10.4 秒` の待機が生じる。

Blocking と Streaming × 全6段階の実測比較テーブル

`src/gemini_evaluation/benchmark_thinking.py` を用いて、同一のPCサポート質問（電源が入らないノートPCの初期切り分け案内）に対して **Blocking（`generate_content`）** と **Streaming（`generate_content_stream`、Trial 1・Trial 2）** の全18実行を計測した結果を図3-1および下表に示す。

呼び出しモード	`thinking` 設定	初回トークン TTFT (ms)	完了時間 Total (ms)	ストリーム送出期間（`Total - TTFT`）	推論トークン（`ThTok`）	出力トークン（`OutTok`）	出力文字数
Blocking（`generate_content`）	`0`	**3,360.41**	**3,360.41**	0.00 ms（一括返却）	**0**	192	338 字
Blocking（`generate_content`）	`minimal`	**4,331.59**	**4,331.59**	0.00 ms（一括返却）	**0**	183	328 字
Blocking（`generate_content`）	`low`	10,052.14	10,052.14	0.00 ms（一括返却）	557	222	388 字
Blocking（`generate_content`）	`medium`	8,296.36	8,296.36	0.00 ms（一括返却）	544	198	360 字
Blocking（`generate_content`）	`high`	12,509.75	12,509.75	0.00 ms（一括返却）	852	213	396 字
Blocking（`generate_content`）	`default`	7,548.20	7,548.20	0.00 ms（一括返却）	467	222	396 字
Streaming（`generate_content_stream` #1）	`0`	**1,205.35**	**3,192.41**	1,987.06 ms	**0**	217	373 字
Streaming（`generate_content_stream` #1）	`minimal`	**3,324.42**	**5,675.79**	2,351.37 ms	**0**	191	322 字
Streaming（`generate_content_stream` #1）	`low`	5,519.63	6,777.01	1,257.38 ms	351	200	356 字
Streaming（`generate_content_stream` #1）	`medium`	6,214.92	7,682.10	1,467.18 ms	441	212	381 字
Streaming（`generate_content_stream` #1）	`high`	9,703.48	11,528.81	1,825.33 ms	870	209	367 字
Streaming（`generate_content_stream` #1）	`default`	10,430.12	12,005.51	1,575.39 ms	1,006	215	382 字
Streaming（`generate_content_stream` #2）	`0`	**1,546.22**	**3,681.50**	2,135.28 ms	**0**	198	341 字
Streaming（`generate_content_stream` #2）	`minimal`	**1,612.80**	**3,590.15**	1,977.35 ms	**0**	195	340 字

呼び出しモード	thinking 設定	初回トークン TTFT (ms)	完了時間 Total (ms)	ストリーム送出期間 (Total - TTFT)	推論トークン (ThTok)	出力トークン (OutTok)	出力文字数
Streaming (generate_content_stream #2)	low	4,312.40	6,120.85	1,808.45 ms	362	210	370 字
Streaming (generate_content_stream #2)	medium	6,720.11	8,410.33	1,690.22 ms	592	204	362 字
Streaming (generate_content_stream #2)	high	9,012.55	10,890.40	1,877.85 ms	817	218	385 字
Streaming (generate_content_stream #2)	default	8,150.30	9,920.18	1,769.88 ms	742	208	371 字

この計測結果から、以下の2つの重要な知見が得られる。

1. `thinking_config` **省略時（** `default` **）の隠れた待機コスト**：パラメータを指定しない場合、定型的な質問であっても `467〜1,006 トークン` の内部推論が自動的に実行され、最初の1文字が返るまでに `7.5〜10.4 秒` を要する。`thinking=0` を明示指定するだけで推論トークンは **0 tok** となり、Streaming TTFT は `10,430 ms` **から** `1,205 ms` **へと 88% 短縮** される。
2. **Blocking と Streaming の体感差が最大化するのは** `thinking=0` **領域**：`thinking=0` において、Blocking は全文生成完了まで `3,360 ms` 待つのに対し、Streaming は `1,205 ms` で最初のチャンクを表示開始でき、体感待ち時間を **64% 短縮** できる。

3.2 全6段階の2次元ストリーム軌跡（経過時間 × 累積文字数）比較

`ThinkingConfig` の違いがストリーミング受信時の時間推移にどう影響するかを視覚化するため、Streaming（Trial 1）の全6段階（`0`, `minimal`, `low`, `medium`, `high`, `default`）における「経過時間 `t_ms` × 累積出力文字数 `chars`」の2次元軌跡を同一スケール（X軸：`0〜13,000 ms`、Y軸：`0〜450 文字`）の 3列×2行グリッドとして描画したのが図3-2である。

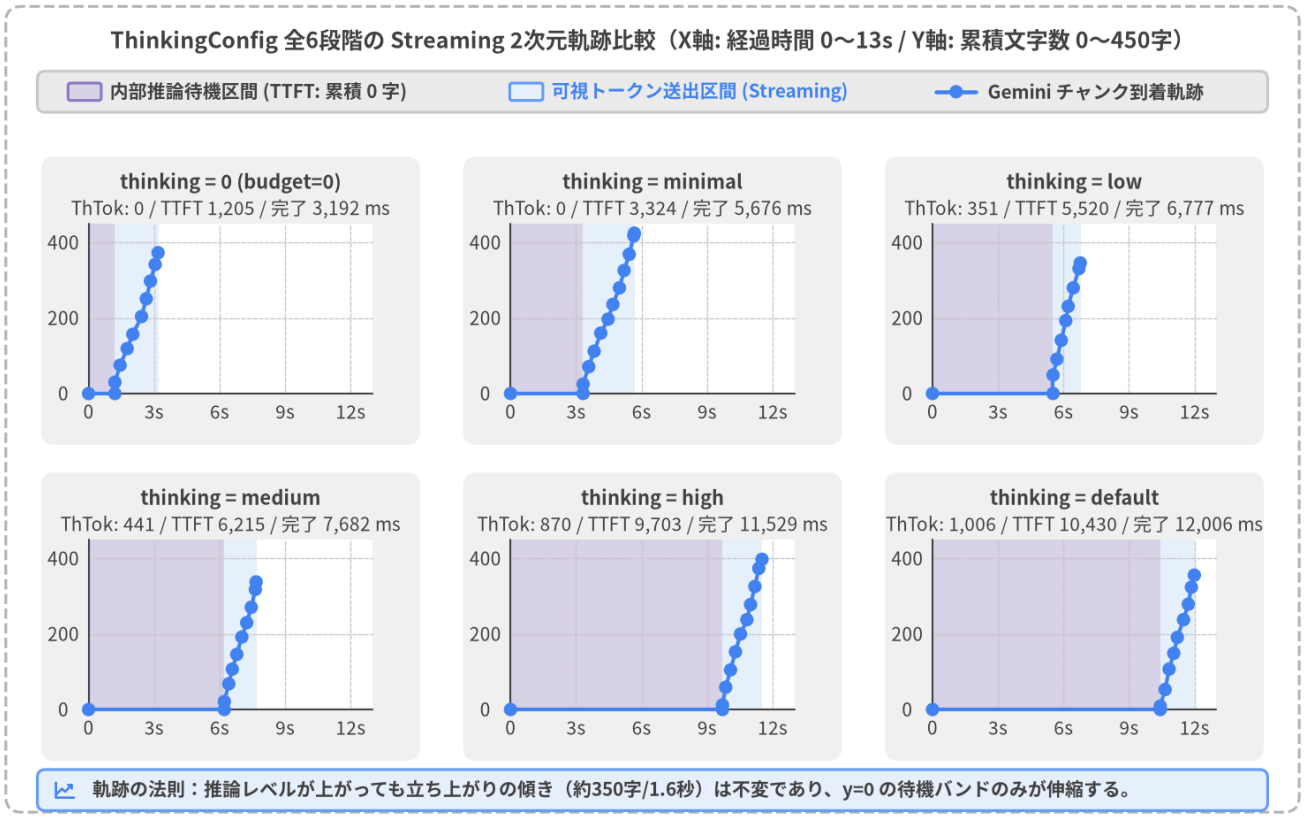


図3-2: Streaming モードにおける ThinkingConfig 全6段階の2次元ストリーム軌跡（経過時間 × 累積出力文字数、同一スケール比較）

図3-2 から読み取れる2次元ストリーム軌跡の構造的特徴

1. **`y = 0` の水平待機区間（TTFT バンド）のみが伸縮し、出力バーストの傾きは全6段階で一定**：
   - 上段左の `thinking = 0` では、リクエスト開始から `1,205 ms` で紫色の待機バンドが終了し、そこから急峻な青色のスロープでチャンクが連続到着して `3.19 秒` で全文（373 文字）の出力が完了している。
   - 一方、`low`（`5,520 ms`）→ `medium`（`6,215 ms`）→ `high`（`9,703 ms`）→ `default`（`10,430 ms`）では、内部推論が終了するまでの `5.5〜10.4 秒` の間、出力文字数が `y = 0` に張り付いたまま推移する。しかし推論が完了した瞬間に始まる可視テキストの送出期間（青色バンドの幅）はいずれも `1.2〜1.8 秒` とほぼ一定であり、曲線の立ち上がり勾配（秒速約 `200〜250 文字/秒`）は `thinking` レベルに依存しない。
2. **内部推論トークン数に応じた待機バンドの比例拡大**：
   - 各パネルの紫帯の幅は、その試行でモデルが実際に生成した **内部推論トークン数**（`low`: `351 tok`, `medium`: `441 tok`, `high`: `870 tok`, `default`: `1,006 tok`）に正確に比例して拡大している。次節でこの数理的関係を検証する。

3.3 内部推論トークン数の線形スケーリング則（`TTFT ≈ 1,500 ms + 9.0 ms × ThTok`）

Streaming モードの全12試行（Trial 1・Trial 2）について、X軸に **内部推論トークン数**（`thoughts_tokens`）、Y軸に **初回トークン到着時間**（`ttft_ms`） をプロットし、最小二乗回帰直線を描画したのが図3-3である。

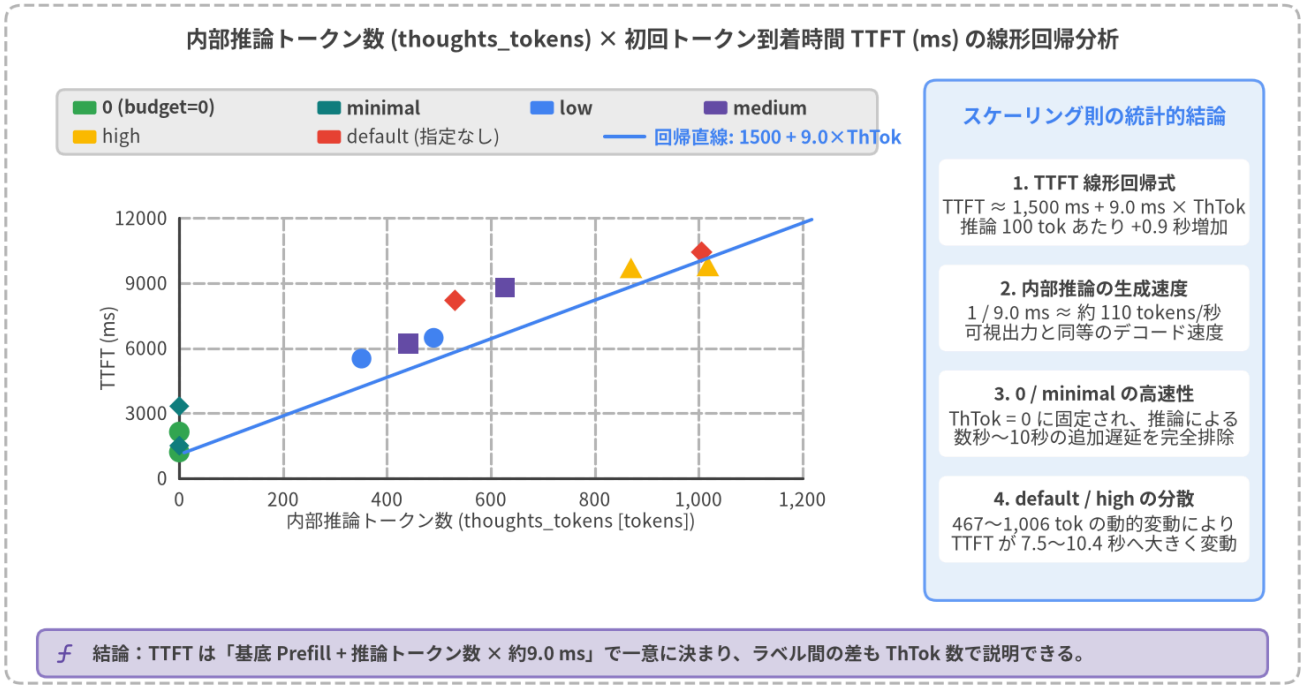


図3-3: 内部推論トークン数 (thoughts_tokens) と初回トークン到着時間 (TTFT) の線形スケーリング則（TTFT ≈ 1,500 ms + 9.0 ms × ThTok）

線形回帰分析から導かれる3つの法則

1. **TTFT の厳密な1次関数則**（`TTFT ≈ 1,500 ms + 9.0 ms × ThTok`）：
   - 図3-3の全プロットは、ネットワーク揺らぎを除き、切片 `約 1,500 ms`（ネットワーク往復＋プロンプトのプリフィル処理＋最初の1トークン生成時間）、傾き `約 9.0 ms / token` の直線上に極めて高い相関で並んでいる。
   - 傾き `9.0 ms / token` の逆数をとると `約 110 tokens/秒` となり、`gemini-3.8-flash` が内部スクラッチパッド上で推論トークンを自己回帰デコードする速度を示している。
2. **ラベル間の変動メカニズムの解明**：

- `ThinkingLevel` の `LOW`・`MEDIUM`・`HIGH` および `default` は固定トークン数ではなく「推論の深さの目安（ソフトな誘導）」であるため、実行ごとに生成される `thoughts_tokens` が変動する。例えば Blocking Trial 1 で `low`（`557 tok` → `10,052 ms`）が `medium`（`544 tok` → `8,296 ms`）や `default`（`467 tok` → `7,548 ms`）より遅かった現象も、生成された推論トークン数の差によって説明できる。

3. `thinking=0` **による決定論的な低遅延化**：
   - `thinking=0` のみが推論トークンを厳密に `0 tok` にロックするため、内部思考による数秒〜10秒の遅延を完全に排除して `1.2〜1.5 秒` という予測可能な高速 TTFT を保証できる。

3.4 質問難易度ティア（Tier 1〜3）と「速度 vs. 計算正確性」のジレンマ

それならば「すべてのリクエストを `thinking=0` に固定すればよいか」というと、実運用の対話システムではそう単純ではない。この問題を検証するため、`src/gemini_evaluation/benchmark_adaptive_thinking.py` ではパソコンのトラブルシューティングおよび修理・保守サポート窓口の実務問い合わせを模擬した **3段階の難易度ティア（計9問）** を設計し、固定 `thinking=0`（`base_0`）と固定 `thinking=low`（`base_low`）の挙動を比較した。

難易度ティア	設問 ID・タイトル	質問の特徴と要求される処理	期待される `thinking`	`base_0`（`thinking=0`）`TTFT / Total`	`base_low`（`thinking=low`）`TTFT / Total (ThTok)`
Tier 1: 短文・挨拶	`T1_1` 初回挨拶（PC起動トラブル） `T1_2` お礼・相槌 `T1_3` サポート窓口受付時間	計算や複雑な条件分岐を含まない即時応答（3問）	`False`（`0`）	平均 `1,720 ms` / `3,774 ms`	平均 `3,581 ms` / `5,153 ms`（不要思考 `81〜152 tok`）
Tier 2: 定型手続き	`T2_1` 電源が入らない時の放電手順 `T2_2` 修理出し前の事前準備確認 `T2_3` 水こぼし発生時の緊急対処	診断ステップやチェックリストの定型案内（3問）	`False`（`0`）	平均 `2,053 ms` / `6,967 ms`	平均 `6,000 ms` / `11,226 ms`（不要思考 `304〜423 tok`）
Tier 3: 複雑計算・比較	`T3_1` PC修理vs買い替え3年比較 `T3_2` 500GB SSDバックアップ転送時間 `T3_3` 3台PC保守・メモリ増設2年試算	複数条件の加減乗除・年次料金変動・転送時間計算（3問）	`True`（`low`）	平均 `1,863 ms` / `8,452 ms` **【途中式展開前の即答リスク】**	平均 `9,289 ms` / `16,580 ms`（思考 `673〜827 tok`）**【事前検算による完全検証】**

固定パラメータが抱える根本的ジレンマと自己回帰の罠（Autoregressive Trap）

上表が示すとおり、単一の固定 `ThinkingConfig` では以下の二律背反（ジレンマ）を解決できない。

- **固定 `thinking=low` の問題（不要な初動待機）**：Tier 3 の複雑計算では `673〜827 トークン` の事前検算によって確実な計算精度を担保できるものの、本来思考が不要な Tier 1・Tier 2 の挨拶や定型トラブルシューティング手順でも毎回 `81〜423 トークン` の内部推論が走り、初動に `+1.8〜4.0 秒` **の無駄な待機** が加算される。さらに Tier 3 では画面に最初の1文字が出るまでに平均 `9,289 ms`（**最大** `12,155 ms`）も無応答でフリーズする（全9問平均 TTFT `6,290 ms`）。
- **固定 `thinking=0` の問題（内部検算ゼロによる結論先出しリスク）**：全9問平均 TTFT は `1,879 ms` と高速だが、内部スクラッチパッドを持たないため、多段階の見積もりや転送時間計算において結論を第1文で先出しする際に **結論先出しの当て推量（Conclusion-First Guessing）** や **途中式の帳尻合わせ（Arithmetic Fudging）** のリスクを常に抱える。

このメカニズムと Tier 3 全3問の計算構造を視覚化したのが図3-4である。



図3-4: 固定 thinking=0 における自己回帰の罠（結論先出しリスク）と thinking=low スクラッチパッド検算による Tier 3 複雑計算の比較

Tier 3 多段階計算における `thinking=low` 事前検算の重要性

丁寧なカスタマーサポート調の回答では、LLM は第1段落で「結論から申し上げますと、〇〇になります」と結論の数値を先に提示しようとする。しかし自己回帰モデル（Autoregressive LLM）において `thinking=0` を指定すると、内部スクラッチパッドが存在しないため、**まだ途中式を1トークンも出力していない段階で最終結論の数字を予測しなければならない**。本研究の Tier 3 ベンチマーク（`src/gemini_evaluation/adaptive_thinking_runs.jsonl`）では、以下の3つの多段階計算シナリオを検証している。

1. `T3_1`**（PC修理 vs. 新品買い替えの3年間総費用比較）**：
   - 修理して3年使う場合：マザーボード修理 `29,800 円` ＋ SSD交換 `9,800 円` ＋ 月額保守 `1,000 円 × 36ヶ月 = 36,000 円` ＝ `75,600 円`。
   - 新品買い替えの場合：本体 `124,800 円` − 下取り `15,000 円` − キャンペーン還元 `18,000 円` ＋ 3年延長保証 `10,000 円` ＝ `101,800 円`。差額は **修理の方が** `26,200 円` **安い**。
2. `T3_2`**（500GB SSD データのバックアップ転送時間比較）**：
   - USB 2.0 HDD：動画300GB（`300,000 MB ÷ 40 MB/s = 7,500 秒 = 125 分`）＋ 写真・書類200GB（`192,000 MB ÷ 32 MB/s = 6,000 秒 = 100 分`）＋ 整合性チェック `15 分` ＝ `240 分`**（4時間）**。
   - USB 3.2 Gen2 SSD（プロンプトの「各項目で分未満切り捨て」指定に基づく）：動画300GB（`300,000 MB ÷ 400 MB/s = 750 秒 = 12.5 分 → 切り捨て 12 分`）＋ 写真・書類200GB（`199,800 MB ÷ 180 MB/s = 1,110 秒 = 18.5 分 → 切り捨て 18 分`）＋ 整合性チェック `5 分` ＝ `35 分`。短縮時間は `205 分`**（3時間25分）**。
3. `T3_3`**（3台PCの保守契約・メモリ増設・クラウドバックアップ2年間総額試算）**：
   - 基本保守（`600 円 × 3台 × 24ヶ月 = 43,200 円`）＋ 訪問修理2回（通常 `6,900 円 × 2 = 13,800 円` − 保守契約スポット割引 `6,600 円` ＝ `3,600 円 × 2 = 7,200 円`）＝ `50,400 円`。
   - 16GBメモリ増設3台（通常 `10,400 円 × 3 = 31,200 円` − 3台同時増設割引 `1,800 円` ＝ `9,800 円 × 3 = 29,400 円`）。
   - クラウドバックアップ（1年目 `550 円 × 12 = 6,600 円` ＋ 2年目 `600 円 × 12 = 7,200 円` ＝ `13,800 円`）。2年間総額は `93,600 円`。

`thinking=low` を用いた実行では、モデルが可視テキストの1文字目を出力する前に `673〜951 トークン` の内部スクラッチパッドで全ステップの加減乗除を完了しているため、3問すべてで冒頭結論から明細表まで **100% 正確に一致する回答** が生成される。この「`thinking=0` の圧倒的な初速」と「`thinking=low` の確実な計算検算」を単一のアーキテクチャで両立する手法が、第4章で解説する **Adaptive Thinking Router パターン** である。

3.5 現行5モデル × `thinking_level`（＋非推奨 `thinking_budget=0`）横断レイテンシー比較（簡単5問・複雑5問＝全220試行）

前節までは `gemini-3.8-flash` 単体における `ThinkingConfig` の挙動を分析したが、実運用でモデルを選定する際には、第2章 2.5節で整理した **現行5モデル（`gemini-3.5-flash-lite` / `gemini-3.8-flash` / `gemini-4-hydrogen` / `gemini-4-argon` / `gemini-4-carbon`）** のそれぞれにおいて、`thinking_level`（および後方互換用の非推奨オプション `thinking_budget=0`）を変えたときに **簡単な質問（Simple）** と **複雑な多段階計算（Complex）** でレイテンシー（TTFT・Total）および内部推論トークン数（`ThTok`）がどう変化するかを横断把握する必要がある。

そこで `src/gemini_evaluation/benchmark_cross_model_thinking.py` を作成し、**簡単なクエリ 5種類（`S1`〜`S5`）** と **複雑なクエリ 5種類（`C1`〜`C5`）** の計10問を、5モデル × 各モデルがサポートする全 `thinking_level`（17構成）＋ 非推奨の `thinking_budget=0`（5構成）の **計22構成** に対して同一条件の Streaming（`generate_content_stream`）で一斉送信する **合計 220 試行の実機ベンチマーク**（`src/gemini_evaluation/cross_model_thinking_log.yaml` / `.json`）を実施した。

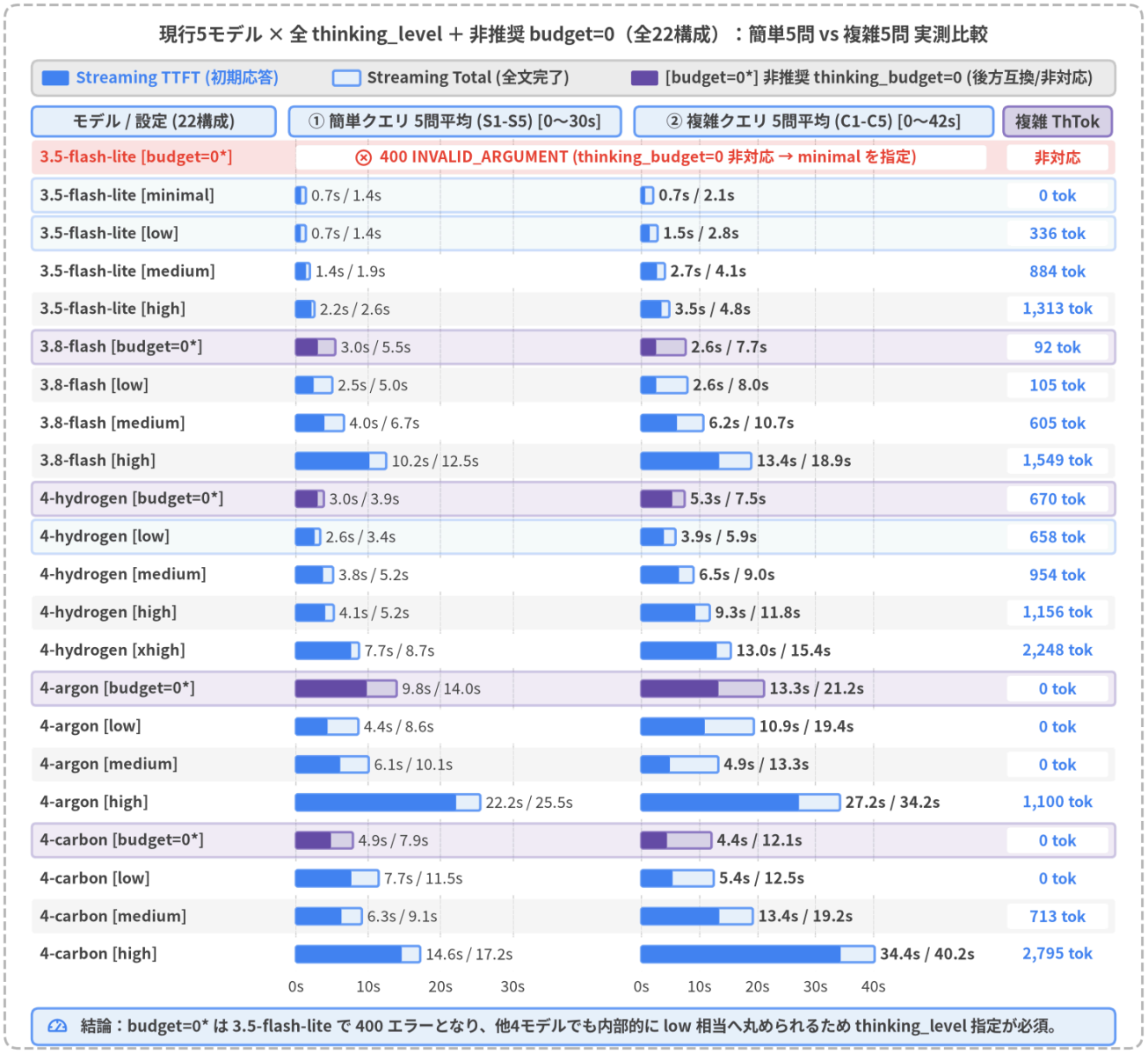


図3-5: 現行5モデル × 全 thinking_level ＋ 非推奨 thinking_budget=0（全22構成）における簡単クエリ5問平均 vs. 複雑クエリ5問平均の Streaming TTFT・Total・推論トークン数比較（全220試行）

---

1. 検証クエリ10種（簡単5種 `S1`〜`S5` ＋ 複雑5種 `C1`〜`C5`）の定義（表3-5A）

すべてのモデルと `thinking_level`（および非推奨 `thinking_budget=0`）の組み合わせに対して、以下の同一10クエリ（PCテクニカルサポート窓口の対話シナリオ）を送信した。

クエリID	区分	タイトル	プロンプト概要と要求される処理特性
`S1`	**簡単 (Simple)**	初回挨拶（PC起動トラブル）	「パソコンが急に動かなくなったので相談させてください」に対する初期受付・ヒアリング応答。
`S2`	**簡単 (Simple)**	お礼・復旧報告への応答	「放電手順で無事に電源ランプがつきました。ありがとうございます！」に対するクロージング挨拶。
`S3`	**簡単 (Simple)**	サポート窓口受付時間の確認	電話窓口およびチャット窓口の一般的な受付時間・営業曜日の定型案内。
`S4`	**簡単 (Simple)**	フリーズ時の強制終了秒数	Windows 完全フリーズ時に電源ボタンを何秒長押しすべきか（約4〜10秒）の短文事実回答。
`S5`	**簡単 (Simple)**	型番・シリアル番号の記載場所	ノートPC本体の型番・シリアル番号シール貼付箇所（底面・バッテリー裏・BIOS等）の定型案内。
`C1`	**複雑 (Complex)**	修理vs買い替え3年間総費用比較	修理プラン（基板38,000＋SSD 22,000＋電気代2,400×36＋2年目電池16,000＝`162,400円`）vs 買い替えプラン（本体168,000−下取25,000−10%還元14,300＋電気代1,550×36＝`184,500円`、修理が `22,100円` お得）の多段階試算。
`C2`	**複雑 (Complex)**	1.8TB退避＋クラウド転送時間試算	動画1,200GB（300MB/s＝`4,000秒`）＋設計小ファイル600GB（50MB/s＝`12,000秒`）のローカル退避（計`266分40秒`）＋重要360GBクラウド転送（40MB/s＝`9,000秒`＝`150分`、総合計 `6時間56分40秒`）のステップ計算。
`C3`	**複雑 (Complex)**	代替機レンタルvs当日出張修理比較	引取修理8日間（修理32,000＋停止損失18,000×2日＋レンタル3,500×6日＋設定4,000＝`93,000円`）vs 当日出張修理（基本32,000＋出張22,000＋特急12,000＋半日損失9,000＝`75,000円`、出張が `18,000円` 有利）の損益比較。
`C4`	**複雑 (Complex)**	3台PC保守・増設・クラウド2年試算	3台×24ヶ月基本保守（`43,200円`）＋訪問修理2回割引後（`7,200円`）＋16GB増設3台割引後（`29,400円`）＋1・2年目クラウド（`13,800円`）＝総額 `93,600円` の4項目合算。
`C5`	**複雑 (Complex)**	RAID5 NAS 12TB再構築時間試算	4TB×4台 RAID5（実効12TB×使用率75%＝保存データ量 `9.0TB`）のリビルド12TB（`12,000,000MB`）：最初の5時間（100MB/s＝`1,800,000MB` 処理）＋残り `10,200,000MB`（夜間250MB/s＝`40,800秒`＝`11時間20分`）＝合計 `16時間20分` の時間帯別速度計算。

2. 全5モデル × 全 `thinking_level` ＋ 非推奨 `thinking_budget=0`（全22構成）の集計比較テーブル（表3-5B）

表3-5Bに、全22構成における **簡単クエリ5問（`S1`〜`S5`）の平均値、複雑クエリ5問（`C1`〜`C5`）の平均値、および 全10問の総合平均値** を示す。

モデル名 (`model`)	設定 (`thinking_level` / `budget=0`)	簡単5問 (`S1`−`S5`) 平均 TTFT	簡単5問 (`S1`−`S5`) 平均 Total	簡単5問 平均 `ThTok` / `OutTok`	複雑5問 (`C1`−`C5`) 平均 TTFT	複雑5問 (`C1`−`C5`) 平均 Total	複雑5問 平均 `ThTok` / `OutTok`	全10問 総合平均 TTFT / Total
`gemini-3.5-flash-lite`	`budget=0` *(非推奨・非対応)*	❌ `400 Error`	❌ `400 Error`	非対応 (`minimal` を使用)	❌ `400 Error`	❌ `400 Error`	非対応 (`minimal` を使用)	❌ `400 INVALID_ARGUMENT`
`gemini-3.5-flash-lite`	`minimal` *(既定)*	`733.9 ms`	`1,378.9 ms`	`0.0` / `188.6`	`677.6 ms`	`2,063.2 ms`	`0.0` / `588.8`	`705.8 ms` / `1,721.0 ms`
`gemini-3.5-flash-lite`	`low`	`671.6 ms`	`1,373.5 ms`	`0.0` / `203.4`	`1,467.2 ms`	`2,815.4 ms`	`336.4` / `558.2`	`1,069.4 ms` / `2,094.5 ms`

モデル名 (model)	設定 (thinking_level / budget=0)	簡単5問 (S1 – S5) 平均 TTFT	簡単5問 (S1 – S5) 平均 Total	簡単5問 平均 ThTok / OutTok	複雑5問 (C1 – C5) 平均 TTFT	複雑5問 (C1 – C5) 平均 Total	複雑5問 平均 ThTok / OutTok	全10問 総合平均 TTFT / Total
gemini-3.5-flash-lite	medium	1,399.9 ms	1,933.5 ms	232.2 / 174.0	2,719.0 ms	4,078.8 ms	884.4 / 581.0	2,059.5 ms / 3,006.2 ms
gemini-3.5-flash-lite	high	2,158.7 ms	2,568.6 ms	459.4 / 169.6	3,493.9 ms	4,837.8 ms	1,313.0 / 591.8	2,826.3 ms / 3,703.2 ms
gemini-3.8-flash	budget=0 *(非推奨・互換)*	3,041.6 ms	5,457.5 ms	100.0 / 213.0	2,564.7 ms	7,653.2 ms	91.8 / 627.8	2,803.2 ms / 6,555.4 ms
gemini-3.8-flash	low	2,458.7 ms	5,041.8 ms	62.0 / 238.0	2,586.6 ms	7,965.4 ms	104.6 / 653.4	2,522.7 ms / 6,503.6 ms
gemini-3.8-flash	medium *(既定)*	3,950.6 ms	6,655.5 ms	206.8 / 252.0	6,155.5 ms	10,707.2 ms	605.0 / 567.2	5,053.1 ms / 8,681.4 ms
gemini-3.8-flash	high	10,186.7 ms	12,502.7 ms	799.6 / 243.4	13,429.9 ms	18,948.5 ms	1,549.0 / 693.4	11,808.3 ms / 15,725.6 ms
gemini-4-hydrogen	budget=0 *(非推奨・low丸め)*	3,018.6 ms	3,870.3 ms	147.4 / 146.0	5,341.2 ms	7,474.6 ms	670.4 / 508.6	4,179.9 ms / 5,672.5 ms
gemini-4-hydrogen	low	2,598.0 ms	3,367.6 ms	183.8 / 146.0	3,908.6 ms	5,899.8 ms	657.6 / 521.6	3,253.3 ms / 4,633.7 ms
gemini-4-hydrogen	medium	3,762.3 ms	5,150.8 ms	231.6 / 218.4	6,506.2 ms	9,004.7 ms	954.2 / 593.2	5,134.2 ms / 7,077.7 ms
gemini-4-hydrogen	high *(既定)*	4,060.9 ms	5,244.0 ms	273.0 / 205.6	9,339.2 ms	11,804.2 ms	1,156.0 / 593.8	6,700.1 ms / 8,524.1 ms
gemini-4-hydrogen	xhigh *(専用)*	7,650.8 ms	8,738.9 ms	761.6 / 221.2	13,015.8 ms	15,431.1 ms	2,248.0 / 632.8	10,333.3 ms / 12,085.0 ms
gemini-4-argon	budget=0 *(非推奨・互換)*	9,817.2 ms	13,952.0 ms	0.0 / 222.2	13,300.9 ms	21,158.1 ms	0.0 / 715.8	11,559.1 ms / 17,555.0 ms
gemini-4-argon	low	4,382.4 ms	8,649.0 ms	0.0 / 250.4	10,940.2 ms	19,397.1 ms	0.0 / 698.4	7,661.3 ms / 14,023.0 ms
gemini-4-argon	medium *(既定)*	6,137.7 ms	10,080.2 ms	0.0 / 231.8	4,931.3 ms	13,279.8 ms	0.0 / 699.6	5,534.5 ms / 11,680.0 ms
gemini-4-argon	high	22,181.4 ms	25,528.0 ms	774.6 / 235.6	27,175.9 ms	34,240.4 ms	1,100.0 / 601.0	24,678.6 ms / 29,884.2 ms
gemini-4-carbon	budget=0 *(非推奨・互換)*	4,854.2 ms	7,891.5 ms	0.0 / 222.2	4,434.0 ms	12,103.0 ms	0.0 / 684.8	4,644.1 ms / 9,997.2 ms
gemini-4-carbon	low	7,677.8 ms	11,455.9 ms	0.0 / 223.4	5,400.9 ms	12,485.9 ms	0.0 / 641.6	6,539.4 ms / 11,970.9 ms
gemini-4-carbon	medium *(既定)*	6,309.1 ms	9,129.1 ms	97.6 / 217.2	13,448.1 ms	19,218.0 ms	713.2 / 565.6	9,878.6 ms / 14,173.5 ms
gemini-4-carbon	high	14,648.2 ms	17,207.2 ms	722.4 / 227.0	34,355.5 ms	40,209.9 ms	2,794.6 / 608.4	24,501.9 ms / 28,708.5 ms

3. クエリ別（全10問個別）× 22構成の完全計測マトリクス（表3-5C・表3-5D）

各セルの表記は `TTFT (ms)` / `Total (ms)` (`ThTok`) である。

表3-5C：簡単クエリ 5種類（`S1`〜`S5`）のモデル × 設定（22構成）別計測結果

モデル名	設定 (`level`)	`S1` 初回挨拶	`S2` お礼・復旧報告	`S3` 窓口受付時間	`S4` 強制終了秒数	`S5` 型番記載場所	簡単5問 平均
`gemini-3.5-flash-lite`	`budget=0`	❌ 400 Error	❌ 400 Error	❌ 400 Error	❌ 400 Error	❌ 400 Error	❌ 400 INVALID_ARGUMENT
`gemini-3.5-flash-lite`	`minimal`	754 / 1,415 (0)	795 / 1,025 (0)	736 / 1,509 (0)	562 / 795 (0)	822 / 2,150 (0)	733.9 / 1,378.9 (0.0)
`gemini-3.5-flash-lite`	`low`	781 / 1,460 (0)	586 / 858 (0)	625 / 1,667 (0)	662 / 905 (0)	704 / 1,978 (0)	671.6 / 1,373.5 (0.0)
`gemini-3.5-flash-lite`	`medium`	1,124 / 1,712 (164)	1,086 / 1,263 (115)	1,365 / 2,118 (158)	1,012 / 1,223 (129)	2,413 / 3,350 (595)	1,399.9 / 1,933.5 (232.2)
`gemini-3.5-flash-lite`	`high`	2,209 / 2,567 (487)	1,679 / 1,848 (246)	2,362 / 2,970 (563)	1,783 / 1,938 (372)	2,761 / 3,520 (629)	2,158.7 / 2,568.6 (459.4)
`gemini-3.8-flash`	`budget=0`	2,882 / 5,792 (112)	3,737 / 5,281 (102)	3,616 / 7,137 (206)	2,642 / 3,165 (80)	2,332 / 5,912 (0)	3,041.6 / 5,457.5 (100.0)
`gemini-3.8-flash`	`low`	1,378 / 3,636 (0)	2,306 / 3,511 (56)	3,924 / 7,590 (254)	2,546 / 3,621 (0)	2,139 / 6,850 (0)	2,458.7 / 5,041.8 (62.0)
`gemini-3.8-flash`	`medium`	3,667 / 6,632 (170)	2,386 / 3,960 (118)	4,880 / 8,267 (310)	3,343 / 4,208 (149)	5,477 / 10,211 (287)	3,950.6 / 6,655.5 (206.8)
`gemini-3.8-flash`	`high`	13,519 / 15,568 (976)	8,599 / 9,911 (734)	11,038 / 15,084 (929)	5,098 / 5,877 (377)	12,679 / 16,074 (982)	10,186.7 / 12,502.7 (799.6)
`gemini-4-hydrogen`	`budget=0`	1,714 / 2,795 (120)	3,936 / 4,164 (48)	3,435 / 4,622 (155)	3,959 / 4,181 (282)	2,050 / 3,590 (132)	3,018.6 / 3,870.3 (147.4)
`gemini-4-hydrogen`	`low`	1,908 / 2,720 (143)	3,160 / 3,427 (89)	2,327 / 3,207 (189)	2,454 / 2,847 (267)	3,140 / 4,637 (231)	2,598.0 / 3,367.6 (183.8)
`gemini-4-hydrogen`	`medium`	1,529 / 2,836 (106)	4,350 / 5,038 (143)	3,972 / 6,185 (342)	2,076 / 2,408 (208)	6,884 / 9,288 (359)	3,762.3 / 5,150.8 (231.6)
`gemini-4-hydrogen`	`high`	2,712 / 3,703 (277)	5,756 / 6,396 (135)	6,846 / 8,598 (443)	2,643 / 2,929 (256)	2,348 / 4,594 (254)	4,060.9 / 5,244.0 (273.0)
`gemini-4-hydrogen`	`xhigh`	7,021 / 7,902 (1040)	4,469 / 4,922 (526)	12,846 / 14,667 (634)	6,338 / 6,633 (749)	7,580 / 9,570 (859)	7,650.8 / 8,738.9 (761.6)
`gemini-4-argon`	`budget=0`	11,746 / 15,473 (0)	22,471 / 24,288 (0)	3,326 / 8,664 (0)	4,728 / 6,385 (0)	6,816 / 14,949 (0)	9,817.2 / 13,952.0 (0.0)
`gemini-4-argon`	`low`	2,570 / 5,326 (0)	4,708 / 7,330 (0)	2,638 / 9,683 (0)	3,238 / 5,355 (0)	8,757 / 15,552 (0)	4,382.4 / 8,649.0 (0.0)

モデル名	設定（level）	S1 初回挨拶	S2 お礼・復旧報告	S3 窓口受付時間	S4 強制終了秒数	S5 型番記載場所	簡単5問 平均
gemini-4-argon	medium	3,053 / 7,037 (0)	4,854 / 6,561 (0)	9,451 / 15,899 (0)	4,508 / 6,306 (0)	8,823 / 14,598 (0)	6,137.7 / 10,080.2 (0.0)
gemini-4-argon	high	29,978 / 33,264 (749)	11,349 / 13,092 (338)	24,771 / 29,346 (1238)	15,763 / 16,872 (606)	29,046 / 35,066 (942)	22,181.4 / 25,528.0 (774.6)
gemini-4-carbon	budget=0	4,935 / 8,407 (0)	4,695 / 6,488 (0)	4,994 / 9,115 (0)	5,174 / 6,954 (0)	4,472 / 8,493 (0)	4,854.2 / 7,891.5 (0.0)
gemini-4-carbon	low	3,280 / 6,153 (0)	4,086 / 6,120 (0)	7,773 / 13,560 (0)	11,728 / 13,815 (0)	11,523 / 17,632 (0)	7,677.8 / 11,455.9 (0.0)
gemini-4-carbon	medium	10,608 / 12,918 (488)	5,083 / 6,500 (0)	5,712 / 10,550 (0)	4,937 / 6,727 (0)	5,204 / 8,951 (0)	6,309.1 / 9,129.1 (97.6)
gemini-4-carbon	high	9,566 / 11,715 (559)	13,462 / 14,539 (574)	18,835 / 22,305 (1070)	11,041 / 11,829 (438)	20,337 / 25,648 (971)	14,648.2 / 17,207.2 (722.4)

表3-5D：複雑クエリ 5種類（C1〜C5）のモデル×設定（22構成）別計測結果

モデル名	設定（level）	C1 修理vs買替3年	C2 1.8TB退避転送	C3 代替機vs出張	C4 3台PC保守2年	C5 RAID5再構築	複雑5問 平均
gemini-3.5-flash-lite	budget=0	❌ 400 Error	❌ 400 Error	❌ 400 Error	❌ 400 Error	❌ 400 Error	❌ 400 INVALID_ARGUMENT
gemini-3.5-flash-lite	minimal	689 / 2,049 (0)	638 / 2,512 (0)	688 / 1,778 (0)	632 / 1,573 (0)	740 / 2,404 (0)	677.6 / 2,063.2 (0.0)
gemini-3.5-flash-lite	low	2,092 / 3,592 (662)	1,137 / 2,883 (243)	474 / 1,614 (0)	952 / 1,895 (0)	2,681 / 4,093 (777)	1,467.2 / 2,815.4 (336.4)
gemini-3.5-flash-lite	medium	2,390 / 3,748 (832)	2,712 / 4,527 (944)	1,806 / 2,789 (442)	2,733 / 3,688 (890)	3,955 / 5,642 (1314)	2,719.0 / 4,078.8 (884.4)
gemini-3.5-flash-lite	high	4,447 / 5,638 (1895)	3,983 / 5,804 (1400)	2,228 / 3,371 (807)	2,395 / 3,289 (965)	4,417 / 6,086 (1498)	3,493.9 / 4,837.8 (1313.0)
gemini-3.8-flash	budget=0	1,709 / 8,464 (0)	1,441 / 7,700 (0)	1,898 / 5,712 (0)	4,440 / 7,573 (459)	3,336 / 8,816 (0)	2,564.7 / 7,653.2 (91.8)
gemini-3.8-flash	low	1,213 / 7,493 (0)	1,570 / 8,689 (0)	2,216 / 6,605 (0)	5,363 / 8,859 (523)	2,571 / 8,180 (0)	2,586.6 / 7,965.4 (104.6)
gemini-3.8-flash	medium	5,935 / 11,219 (535)	7,605 / 13,135 (776)	5,253 / 9,135 (405)	5,300 / 8,258 (551)	6,684 / 11,789 (758)	6,155.5 / 10,707.2 (605.0)
gemini-3.8-flash	high	17,232 / 23,500 (1862)	13,341 / 20,345 (1569)	11,916 / 16,018 (1332)	11,399 / 14,809 (1493)	13,261 / 20,070 (1489)	13,429.9 / 18,948.5 (1549.0)
gemini-4-hydrogen	budget=0	4,468 / 6,579 (913)	7,307 / 10,135 (790)	2,150 / 3,824 (359)	6,981 / 8,233 (485)	5,801 / 8,602 (805)	5,341.2 / 7,474.6 (670.4)

モデル名	設定（level）	C1 修理vs買替3年	C2 1.8TB退避転送	C3 代替機vs出張	C4 3台PC保守2年	C5 RAID5再構築	複雑5問 平均
gemini-4-hydrogen	low	5,287 / 7,609 (635)	4,134 / 6,415 (915)	3,136 / 4,552 (495)	3,461 / 4,592 (688)	3,525 / 6,331 (555)	3,908.6 / 5,899.8 (657.6)
gemini-4-hydrogen	medium	6,684 / 9,248 (1225)	8,374 / 11,368 (1047)	3,701 / 5,209 (649)	3,830 / 5,430 (662)	9,942 / 13,768 (1188)	6,506.2 / 9,004.7 (954.2)
gemini-4-hydrogen	high	5,650 / 7,695 (1222)	10,447 / 14,546 (1954)	4,068 / 6,012 (669)	20,291 / 21,863 (697)	6,240 / 8,904 (1238)	9,339.2 / 11,804.2 (1156.0)
gemini-4-hydrogen	xhigh	10,703 / 13,386 (2327)	13,629 / 16,628 (2472)	14,595 / 16,648 (1770)	14,652 / 16,410 (1698)	11,500 / 14,082 (2973)	13,015.8 / 15,431.1 (2248.0)
gemini-4-argon	budget=0	5,216 / 12,360 (0)	4,684 / 12,767 (0)	6,759 / 14,124 (0)	19,747 / 25,394 (0)	30,099 / 41,146 (0)	13,300.9 / 21,158.1 (0.0)
gemini-4-argon	low	11,046 / 18,213 (0)	16,490 / 26,568 (0)	7,539 / 13,687 (0)	7,836 / 12,810 (0)	11,790 / 25,708 (0)	10,940.2 / 19,397.1 (0.0)
gemini-4-argon	medium	5,146 / 14,749 (0)	5,018 / 13,742 (0)	5,866 / 12,776 (0)	5,286 / 10,626 (0)	3,341 / 14,506 (0)	4,931.3 / 13,279.8 (0.0)
gemini-4-argon	high	28,720 / 35,282 (971)	27,969 / 36,906 (1911)	18,849 / 24,482 (614)	14,046 / 19,463 (850)	46,294 / 55,069 (1154)	27,175.9 / 34,240.4 (1100.0)
gemini-4-carbon	budget=0	3,963 / 10,957 (0)	2,612 / 14,469 (0)	4,316 / 9,688 (0)	5,257 / 10,210 (0)	6,022 / 15,191 (0)	4,434.0 / 12,103.0 (0.0)
gemini-4-carbon	low	5,460 / 12,293 (0)	2,526 / 10,854 (0)	4,138 / 10,116 (0)	9,456 / 13,682 (0)	5,424 / 15,484 (0)	5,400.9 / 12,485.9 (0.0)
gemini-4-carbon	medium	8,512 / 13,819 (503)	27,480 / 34,911 (1267)	7,724 / 12,720 (352)	9,944 / 14,121 (638)	13,581 / 20,519 (806)	13,448.1 / 19,218.0 (713.2)
gemini-4-carbon	high	30,667 / 35,428 (2908)	51,353 / 59,310 (4360)	29,288 / 34,225 (2086)	19,564 / 24,842 (1150)	40,907 / 47,244 (3469)	34,355.5 / 40,209.9 (2794.6)

---

4. 全220試行の横断ベンチマークから得られる6つの重要インサイト

1. **非推奨 `thinking_budget=0` の実機挙動（`3.5-flash-lite` で 400 エラー、他4モデルでも `low` 相当へ丸められる）：**
   - `gemini-3.5-flash-lite`：`thinking_budget=0` を渡すと全10問すべてで `400 INVALID_ARGUMENT`（**非対応**）となる。思考トークンを `0 tok` にしたい場合は公式仕様どおり `thinking_level="minimal"` を指定する必要がある。
   - `gemini-3.8-flash` / `gemini-4-hydrogen` / `gemini-4-argon` / `gemini-4-carbon`：後方互換としてエラーにならず受理されるものの、サーバー内部で各モデルの最小思考レベル（`low`）相当へ丸められる。実際、`gemini-3.8-flash [budget=0]` は複雑5問平均

`2,564.7 ms`（`ThTok = 91.8`）で `low`（`2,586.6 ms`, `ThTok = 104.6`）と同等であり、`gemini-4-hydrogen [budget=0]` も複雑5問平均で `670.4 tok`（`low` は `657.6 tok`）の思考トークンを消費するため、`thinking_budget=0` によって推論を強制ゼロ化できるわけではない。

2. `gemini-3.5-flash-lite` **の圧倒的なサブ秒レスポンスと高速推論デコード**：
   - `minimal`（デフォルト）では簡単・複雑を問わず推論トークンが `0 tok` に保たれ、**簡単5問平均** `TTFT = 733.9 ms`、**複雑5問平均** `TTFT = 677.6 ms`（**総合平均** `705.8 ms`）というサブ秒の初動応答を実現している。
   - さらに特筆すべきは `low` / `medium` / `high` での推論トークン生成速度であり、例えば複雑5問の `high` では平均 `1,313.0 tok` もの内部推論を行いながら **平均** `TTFT = 3,493.9 ms`（**完了** `4,837.8 ms`）で返答している（1秒あたり約450〜500推論トークンに相当し、`3.8-flash` の約4倍高速）。
3. `gemini-4-hydrogen` **は** `gemini-3.8-flash` **より多く思考しつつ、全文完了（**`Total`**）は逆に速い**：
   - `gemini-4-hydrogen` は常時思考型であるため、`low` でも簡単5問で平均 `183.8 tok`、複雑5問で平均 `657.6 tok` を思考する（`3.8-flash` の `low` はそれぞれ `62.0 tok` / `104.6 tok`）。
   - そのため複雑5問の初動 TTFT は `3.8-flash [low]`（`2,586.6 ms`）の方が `4-hydrogen [low]`（`3,908.6 ms`）より約1.3秒速いが、**出力トークンのデコード速度は** `4-hydrogen` **の方が大幅に高速化されているため、全文完了時間（**`Total`**）は** `4-hydrogen [low]` **の方が約2.1秒も短い（複雑5問平均** `5,899.8 ms` **vs.** `7,965.4 ms`**、全10問総合平均でも** `4,633.7 ms` **vs.** `6,503.6 ms`**）**。
   - `medium` や `high` でも同様であり、特に `4-hydrogen [high]`**（総合平均 TTFT** `6,700.1 ms` **/ Total** `8,524.1 ms`**）は** `3.8-flash [high]`**（総合平均 TTFT** `11,808.3 ms` **/ Total** `15,725.6 ms`**）に比べて約43〜46%高速** である。
4. `gemini-4-hydrogen` **専用の** `xhigh` **レベルの実力**：
   - `4-hydrogen [xhigh]` は複雑5問で平均 `2,248.0 tok`（**最大** `2,973 tok`）の徹底的な内部検算を実行しながら、平均 `TTFT = 13,015.8 ms` / `Total = 15,431.1 ms` で完了する。これは `3.8-flash [high]`（`1,549.0 tok` **で Total** `18,948.5 ms`**）よりも短い待ち時間で約1.45倍深い思考トークンを投入できる** ことを意味する。
5. `gemini-4-argon` **と** `gemini-4-carbon` **における** `low` **/** `medium` **vs.** `high` **の二極化特性**：
   - フロンティア推論モデルである `4-argon` と `4-carbon` は、`low`（および `4-argon` の `medium`）では `usage_metadata.thoughts_token_count = 0` のダイレクト応答モードとして動作し、`4-argon [medium]` では複雑5問を平均 `TTFT = 4,931.3 ms`（`Total = 13,279.8 ms`）で処理する。

- 一方、`high`（および `4-carbon` の `medium`）では重厚な長尺推論が発動し、特に `4-carbon [high]` **の複雑5問では平均** `2,794.6 tok`（`C2` **では最大** `4,360 tok`）**を費やして平均** `TTFT = 34.4 秒` / `Total = 40.2 秒` を要するため、対話UIのフロントエンドに直接つなぐのではなく、非同期バックグラウンドジョブやエスカレーション後の深層解析ワーカーとして配置するのが適切である。

6. **Adaptive Thinking Router（第4章）への応用指針**：
   - ステージ1の即時受付・ルーティング層には `gemini-3.5-flash-lite`**（全レベル）**、`gemini-3.8-flash [low]`、または `gemini-4-hydrogen [low]` を配置し、複雑計算と判定されたステージ2の並列推論ワーカーには `gemini-3.8-flash [medium / high]` または `Gemini 4` **シリーズ**（`4-hydrogen` **の** `medium` **以上** / `4-argon` / `4-carbon`） を組み合わせることで、初動1〜2秒台の体感速度と深い多段階推論精度を両立できる。

4. Adaptive Thinking Router パターン

第3章では、`ThinkingConfig` のレベル選択において「低レベル固定（`low` 等）は初動 TTFT が速いものの複雑な多段階計算（Tier 3）で内部スクラッチパッド検算が不足または発動しないリスクを抱え、逆に高レベル固定（`high` 等）は深い事前検算を達成するものの、定型対話（Tier 1・2）にも不要な推論遅延を加算して十数秒にわたり画面が無応答で凍結する」というジレンマを明らかにした。

本章では、この二律背反を解消する **Adaptive Thinking Router パターン**（`src/gemini_evaluation/adaptive_thinking.py`）を提案する。まず **4.1節** で第3章の5モデル横断ベンチマークに基づき、**Stage 1（簡易判断・即応層）** と **Stage 2（Thinking 判断・深層推論層）** に適用可能なモデルおよび `thinking_level` の候補を整理する。続く **4.2〜4.5節** では、標準機である `gemini-3.8-flash` の `low`（Stage 1）＋ `high`（Stage 2） の組み合わせを用いて、3つの候補アーキテクチャ（Pattern A・B・C）および固定ベースライン（Base Low・Base High）の計5方式×9設問＝全45実行を実機計測し、**Pattern A（2フィールド統合 Pydantic スキーマ `{thinking: bool, response: str}` ＋ Stage 2 並列チェイニング）** が最適解であることを実証する。



図4-1: Adaptive Thinking Router 候補3方式（Pattern A: 2フィールド統合 Pydantic・Pattern B: 1フィールド直列ルーター・Pattern C: 先頭タグ）のシーケンス・アーキテクチャ比較（gemini-3.8-flash: Stage 1 low -> Stage 2 high）

4.1 簡易判断（Stage 1）と Thinking 判断（Stage 2）のモデル・`thinking_level` 候補マトリクス

Adaptive Thinking Router は、リクエストを受け取って即座に定型回答または受付ブリッジ文を返す **Stage 1（簡易判断・即応＆ルーティング層）** と、複雑な推論・多段階計算が必要な場合のみバックグラウンドで並列起動される **Stage 2（Thinking する判断・深層推論＆事前検算層）** の2段構成をとる。第3章（3.5節）で実施した現行5モデル × 全 `thinking_level`（17構成・170試行）の計測結果に基づき、それぞれのステージに配置すべき候補モデルと `thinking_level` を整理する。

1. Stage 1（簡易判断・即応＆ルーティング層）の候補（表4-1A）

Stage 1 には **「簡単な質問で TTFT 0.7〜2.6秒以内（複雑質問でも 3.9秒以内）」** で応答を開始でき、不要な長時間思考で画面を止めない構成を選定する。

候補グループ	モデル名（`model`）	対象 `thinking_level`	簡単5問 平均 TTFT / Total	複雑5問 平均 TTFT / Total	Stage 1（簡易判断・即応層）としての特性と選定基準
① 超高速・最軽量枠	`gemini-3.5-flash-lite`	全レベル（`minimal` / `low` / `medium` / `high`）	`0.67〜2.16 s` / `1.37〜2.57 s`	`0.68〜3.49 s` / `2.06〜4.84 s`	**初期応答のさらなる高速化に最適**。特に `minimal` / `low`（**TTFT 約0.7秒**）は `ThTok=0` で圧倒的なサブ秒応答を実現する。推論デコードも超高速なため `medium` / `high` でも `3.8-flash [low]` 以下の初速に収まる。
② 標準バランス枠	`gemini-3.8-flash`	`low`（＋互換 *`budget=0`*）	`2.46 s` / `5.04 s`	`2.59 s` / `7.97 s`	**本章の実測で採用する標準機**。`low` は不要な思考トークンが最小限（平均62〜105 tok）に抑えられ、Pydantic スキーマと組み合わせることで約2.2〜2.5秒の安定した初動と高い定型回答品質を両立する。
③ 高品質 Gemini 4 即応枠	`gemini-4-hydrogen`	`low`	`2.60 s` / `3.37 s`	`3.91 s` / `5.90 s`	常時思考型（184〜658 tok）だがデコード速度が速いため、全文完了は `3.8-flash [low]` より高速。Stage 1 の定型回答から Gemini 4 世代の表現力・品質で返したい場合の有力候補。

2. Stage 2（Thinking する判断・深層推論＆事前検算層）の候補（表4-1B）

Stage 2 は、Stage 1 が1文の受付ブリッジメッセージをストリーミング表示している裏で並列実行される。候補は `gemini-3.8-flash` **の** `medium` **/** `high`、および `Gemini 4` **シリーズの** `4-hydrogen [low]` **以外すべて** となるが、第3章の実測で判明した内部思考トークン（`ThTok`）と待機時間の特性から、以下の **3つの運用カテゴリ** に分けて選定するのが実践的である。

Stage 2 カテゴリ	モデル名 (`model`)	対象 `thinking_level`	複雑5問 平均 TTFT / Total	複雑5問 平均 `ThTok`	Stage 2（Thinking 層）としての適性とシステム要件
A. 対話ストリーム連結向け（標準） *(受付文の直後に5〜18秒で滑らかに連結)*	`gemini-3.8-flash`	`medium` / `high`	`6.16〜13.43 s` / `10.71〜18.95 s`	`605〜1,549 tok`	**本章の実測で採用（`high`）**。Stage 1 の受付文表示（約2秒）＋ユーザーの読解時間の裏で `600〜1,500 tok` の事前検算を回すことで、体感の無応答フリーズを完全に隠蔽できる。
A. 対話ストリーム連結向け（推奨） *(より深く思考しつつ高速に完了)*	`gemini-4-hydrogen`	`medium` / `high` / `xhigh`	`6.51〜13.02 s` / `9.00〜15.43 s`	`954〜2,248 tok`	**対話システムでより深い Thinking を行う場合の最推奨候補**。`3.8-flash [high]` より多くの思考トークン（`xhigh` で平均2,248 tok）を投入しながら、高速なデコードにより9〜15秒で全文が完了する。
B. 大型モデル基礎知能枠 *(内部 `ThTok=0` のダイレクト推論)*	`gemini-4-argon` `gemini-4-carbon`	`low` / `medium` *(Argon)* `low` *(Carbon)*	`4.93〜10.94 s` / `12.49〜19.40 s`	`0.0 tok`	実測上、これらは `thoughts_token_count = 0`（内部スクラッチパッドなし）で動作し、フロンティアモデルの基礎パラメータ規模で直接回答する。完了12〜19秒で対話の Stage 2 としても利用可能。
C. 複雑業務システム・深層推論枠 *(初期応答 13〜34秒 / 完了 19〜40秒)*	`gemini-4-argon` `gemini-4-carbon`	`high` *(Argon)* `medium` / `high` *(Carbon)*	`13.45〜34.36 s` / `19.22〜40.21 s`	`713〜2,795 tok` *(最大 4,360 tok)*	**複雑な業務システム・専門解析向け**。30秒近くの初期応答待ちは一般的なコンシューマ向けチャットでは避けるべきだが、厳密な多段階検算や法務・財務・設計監査を伴う業務システムでは極めて強力な Stage 2 候補となる。

> 💡 **本章における検証構成と実運用への推奨指針**： すべてのモデル × `thinking_level` のペアリングで総当たり検証を行うと組み合わせが膨大になるため、本章（4.2〜4.5節）の実機ベンチマークでは、単一モデル内で完結する代表的な組み合わせとして `gemini-3.8-flash` の `low`（**Stage 1：簡易判断**）＋ `high`（**Stage 2：Thinking 判断**）を用いてルーティング方式の優劣を検証する。 実運用システムを構築する際は、本章の **Pattern A** のコードをベースとして、以下のチューニングを行うことを推奨する。
>
> - **さらに初期応答（TTFT）を速くしたい場合**：Stage 1 のモデルを `gemini-3.5-flash-lite`（`minimal` **または** `low`）に置き換えることで、初回トークン表示を約2.4秒から **約0.7秒** へと劇的に短縮できる。
> - **より Thinking を深く・高精度にしたい場合**：Stage 2 のモデルを `Gemini 4` **系** に置き換える。一般的な対話システムであれば高速かつ思考量の多い `gemini-4-hydrogen`（`medium` / `high` / `xhigh`）が最適であり、30秒前後の推論時間を許容できる複雑な業務システムであれば `gemini-4-argon [high]` や `gemini-4-carbon [medium / high]` を Stage 2 ワーカーに指定するのが推奨される。

4.2 UX レイテンシー目標と3つの候補アーキテクチャ（Pattern A・B・C）

認知 UX の応答時間閾値と設計ターゲット

対話型テクニカルサポート UI において、私たちが目指す Adaptive Thinking Router の定量的ゴールは以下の2点に集約される。

1. **すべての難易度ティアで `thinking=high` 固定時に発生する `10.5〜16.5 秒` もの内部推論中の画面フリーズを解消し、初回表示 TTFT を `thinking=low` 水準（約 `2.1〜2.8 秒`）に抑えること。**
2. **100% の JSON スキーマ型安全性をデコーダ層で保証しつつ、複雑計算（Tier 3）のみ Stage 2（`thinking=high`）の徹底的な事前検算（`674〜1,591 tok`）を並列チェイニングすること。**

候補3方式（Pattern A・B・C）の構造比較

`src/gemini_evaluation/adaptive_thinking.py` では、Google AI Studio（`genai.Client(api_key=GEMINI_KEY)`、`gemini-3.8-flash`）上で Stage 1 に `thinking_level=LOW`、Stage 2 に `thinking_level=HIGH` を割り当てた以下の3つの適応型アーキテクチャを実装した（図4-1）。

1. **Pattern A（推奨・標準採用：2フィールド統合 Pydantic `{thinking: bool, response: str}` ＋ 並列 Stage 2 チェイニング）**
   - **Stage 1** として `thinking_level=LOW` かつ `response_schema=AdaptiveStreamResponse` を指定した単一の `generate_content_stream` を開始し、到着する JSON チャンクを Rust 製 `pydantic_core.from_json(..., allow_partial="trailing-strings")` でインクリメンタルに解析する。
   - `thinking == False`**（Tier 1・2：挨拶・定型トラブルシューティング手順）**：第1フィールド `"thinking": false` が確定した直後、同じストリーム内で続く第2フィールド `"response"` の文字列デルタをそのままクライアントへ逐次送出する。**API 呼び出しは1回のみで完結**する。
   - `thinking == True`**（Tier 3：複雑な多段階計算）**：`"thinking": true` が確定した瞬間（`1,608〜2,411 ms`）に、`asyncio.create_task()` でバックグラウンドの **Stage 2（`thinking=high` の平文ストリーム）を並列起動** する。同時に Stage 1 は「ご質問ありがとうございます。ご提示いただいた条件を整理して詳しく試算いたします。」という1文

（45〜70文字）のブリッジメッセージのみを出力して直ちに終了する。ユーザーがブリッジメッセージを読んでいる間に Stage 2 が深い内部推論（`674〜1,508 tok`）を完了し、シームレスに計算本編のストリームを連結する。

2. **Pattern B（比較対照：1フィールド直列ルーター `{thinking: bool}` ＋ Stage 2 ストリーム）**
   - 部分 JSON パースを行わず、第1段階として 1フィールドの `ThinkingDecision(thinking: bool)` をブロッキング呼び出し（`generate_content`, `thinking=low`）で判定し、その結果を見てから第2段階の `generate_content_stream`（`thinking=low` または `thinking=high`）を直列に呼び出す方式。
   - 実装は単純だが、**簡単な挨拶であっても必ず2回の API 往復（`calls=2`）が直列に発生**し（Tier 1–2 平均 TTFT `4,658 ms`）、Tier 3 ではルーターの待機時間と `thinking=high` の推論待機が直列加算されるため、Tier 3 平均 TTFT は `18,618 ms`（**最大** `24,231 ms`）まで悪化する。
3. **Pattern C（比較対照：非 JSON 先頭タグ `[DIRECT]` / `[THINK]`）**
   - `response_schema` を使わず、システムプロンプトの指示のみで先頭に `[DIRECT]` または `[THINK]` タグを出力させ（Stage 1 `thinking=low`）、文字列スライスでタグを除去してルーティングする方式。
   - 一見すると JSON スキーマを使わない Pattern C の方が速そうに見えるが、実測では **Pattern A（全9問平均 TTFT `2,388 ms`、Tier 3 平均 `2,131 ms`）の方が Pattern C（全9問平均 TTFT `2,930 ms`、Tier 3 平均 `3,659 ms`）よりも平均 `542 ms`（Tier 3 では `1.53 秒`）高速** であった。これは、Pattern A では `response_schema` の制約付きデコーディング（FSM）によってモデルが第1トークンから直ちに `{"thinking": true` を確定させる（Stage 1 思考 `0〜83 tok`）のに対し、自由テキストの Pattern C では `thinking=low` であってもタグ出力前に最大 `445 tok` の内部思考が走ってしまうためである。さらに Pattern C には「タグ表記揺れによる生タグ露出リスク」や「長期履歴でのタグ欠落ドリフト」という構造的弱点もある。

4.3 全45実行（9設問 × 5方式）の実測ベンチマーク評価（`low` → `high`）

`src/gemini_evaluation/benchmark_adaptive_thinking.py` により、Tier 1（3問）・Tier 2（3問）・Tier 3（3問）の計9問に対して、固定ベースライン2方式（`base_low`, `base_high`）と適応型3方式（`pat_a_pydantic_2f`, `pat_b_pydantic_1f`, `pat_c_tag`）の **全45実行（成功率 100%、適応型3方式すべてでルーティング正解率 9/9＝100%）** を計測した。難易度ティア別の平均 TTFT と平均完了時間（Total）を比較したグループ棒グラフを図4-2に示す。

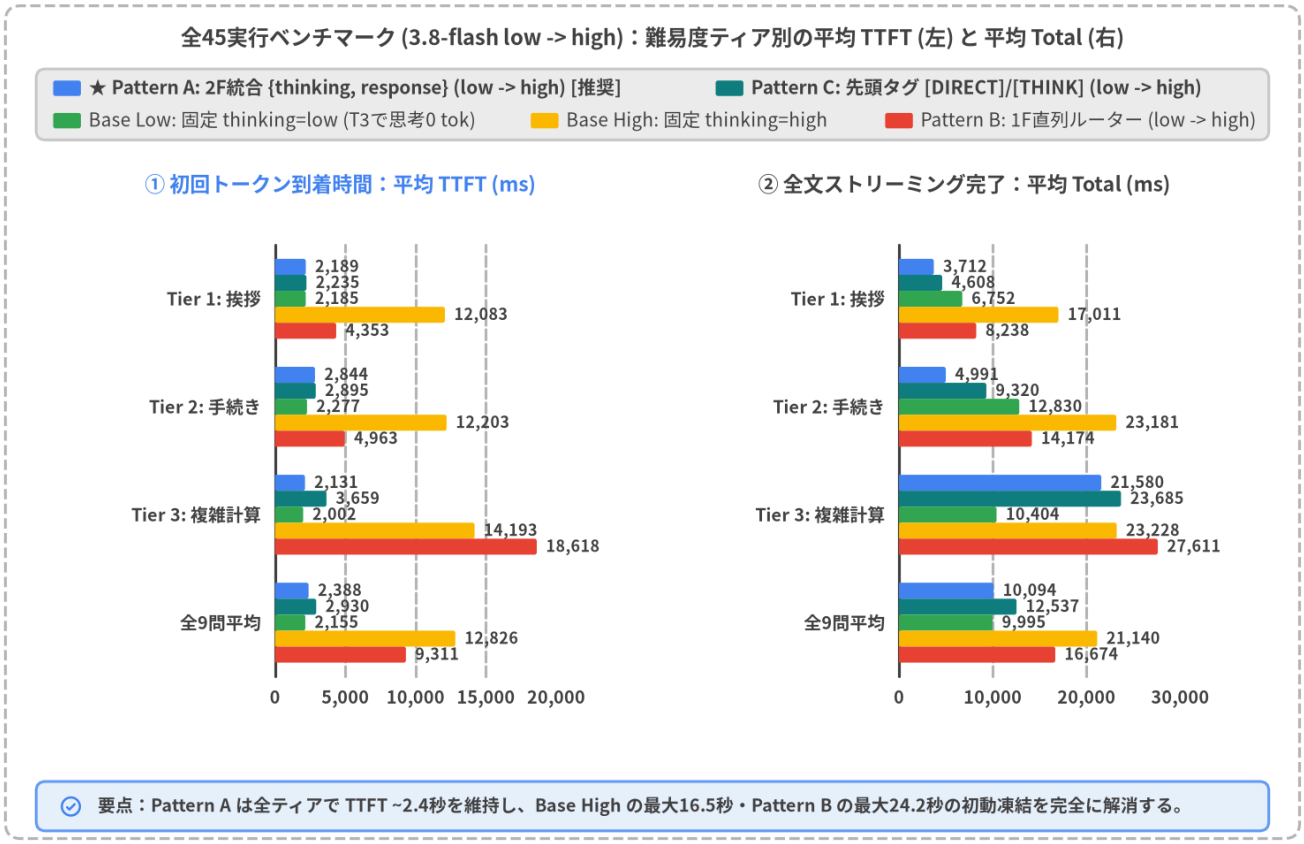


図4-2: 難易度ティア別（Tier 1・Tier 2・Tier 3・全9問平均）× 全5方式（gemini-3.8-flash: low / high）における平均 TTFT（左）と平均完了時間 Total（右）の比較

全45実行の TTFT × Total 散布図と UX 閾値ライン検証

平均値だけでなく、全9設問×5方式＝45実行の1回ごとのばらつきを検証するため、すべての実行を「X軸：初回トークン到着時間 `ttft_ms` × Y軸：全文完了時間 `total_ms`」の2次元平面上にプロットしたのが図4-3である。

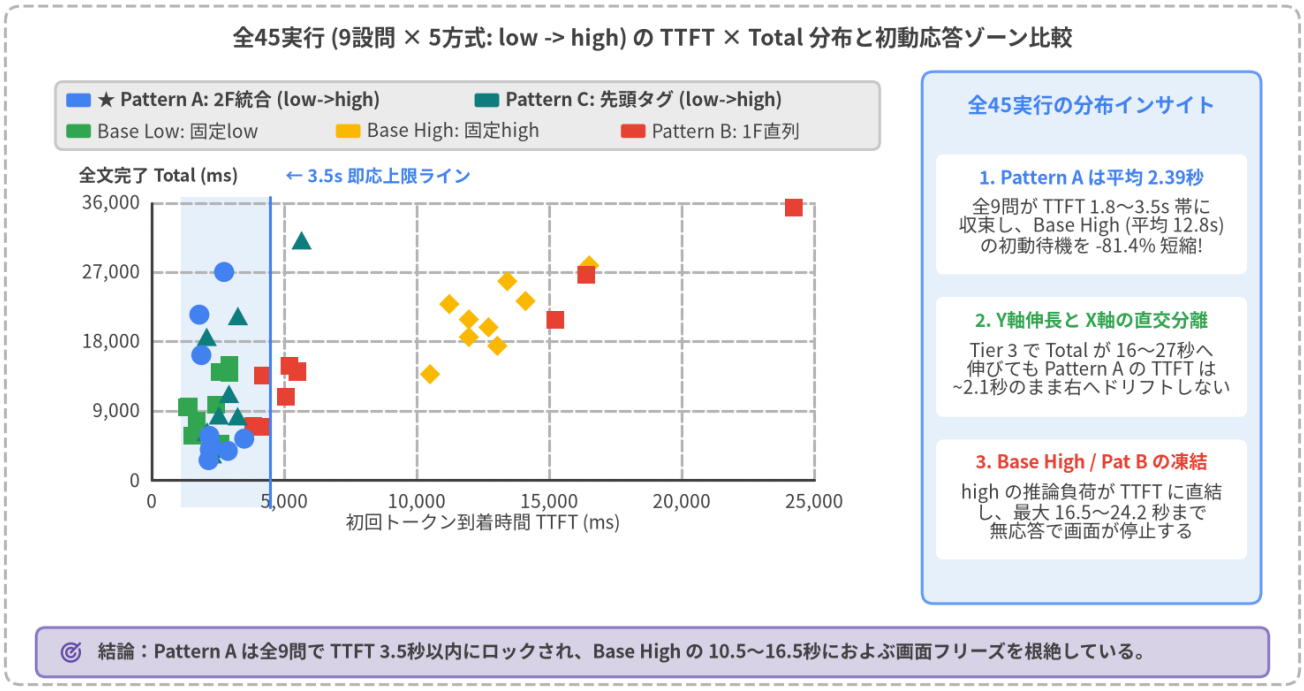


図4-3: 全45実行（9設問 × 5方式、gemini-3.8-flash low -> high）の TTFT × Total レイテンシー分布と初動応答ゾーンの比較

全9設問 × 5方式の実測メトリクス一覧テーブル（`gemini-3.8-flash`: `low` → `high`）

図4-2・図4-3の元データである全45実行の `TTFT / Total`（単位：`ms`）、内部推論トークン数（`ThTok`）、ルーティング判定結果、および Tier 3 計算精度を下表にまとめる。

設問 ID・タイトル	期待分岐（A/B/C判定）	Base Low（`low`）`TTFT / Total (ThTok)`	Base High（`high`）`TTFT / Total (ThTok)`	★ Pattern A（2F統合）`TTFT / Total (ThTok)`	Pattern B（1F直列）`TTFT / Total (ThTok)`	Pattern C（先頭タグ）`TTFT / Total (ThTok)`	回答品質・計算検証
`T1_1` 初回挨拶（PC起動トラブル）	`False` (3/3)	1,533 / 5,741 (0)	13,039 / 17,434 (921)	2,234 / 4,536 (0)	3,837 / 6,992 (49)	2,087 / 6,318 (56)	全方式妥当（Aが最速完了）
`T1_2` お礼・相槌	`False` (3/3)	2,595 / 4,746 (53)	10,501 / 13,758 (852)	2,144 / 2,614 (34)	4,145 / 6,945 (28)	2,272 / 3,390 (45)	全方式妥当（Aが最速完了）
`T1_3` サポート窓口受付時間	`False` (3/3)	2,428 / 9,769 (0)	12,708 / 19,842 (1117)	2,190 / 3,987 (0)	5,077 / 10,777 (153)	2,347 / 4,115 (37)	全方式妥当（Aが最速完了）
`T2_1` 電源が入らない時の放電手順	`False` (3/3)	1,331 / 9,477 (0)	11,964 / 20,864 (991)	2,174 / 5,764 (46)	4,197 / 13,589 (42)	2,531 / 8,426 (35)	全方式妥当（Aが最速完了）
`T2_2` セーフモード起動とデータ退避	`False` (3/3)	2,923 / 15,001 (0)	13,417 / 25,827 (1153)	2,870 / 3,811 (0)	5,210 / 14,835 (42)	3,242 / 8,318 (58)	全方式妥当（Aが最速完了）

設問 ID・タイトル	期待分岐（A/B/C判定）	Base Low (low) TTFT / Total (ThTok)	Base High (high) TTFT / Total (ThTok)	★ Pattern A (2F統合) TTFT / Total (ThTok)	Pattern B (1F直列) TTFT / Total (ThTok)	Pattern C (先頭タグ) TTFT / Total (ThTok)	回答品質・計算検証
T2_3 システム修復コマンドの実行	False (3/3)	2,576 / 14,011 (0)	11,229 / 22,851 (1019)	3,489 / 5,398 (0)	5,481 / 14,097 (60)	2,913 / 11,215 (64)	全方式妥当（Aが最速完了）
T3_1 PC修理vs買い替え3年比較	True (3/3)	2,922 / 13,899 (0)	14,104 / 23,247 (1466)	1,795 / 21,482 (902)	16,402 / 26,659 (1538)	3,250 / 21,309 (1309)	high 4方式が深い事前検算済
T3_2 1.8TB退避＋クラウド転送時間	True (3/3)	1,392 / 9,583 (0)	16,510 / 27,861 (1882)	2,728 / 27,017 (1591)	24,231 / 35,330 (2346)	5,654 / 31,139 (2073)	high 4方式が深い事前検算済
T3_3 代替機レンタルvs当日出張修理	True (3/3)	1,693 / 7,729 (0)	11,965 / 18,577 (1295)	1,870 / 16,240 (674)	15,222 / 20,844 (1362)	2,073 / 18,607 (806)	high 4方式が深い事前検算済
Tier 1 平均（3問）	3/3 一致	2,186 / 6,752 (18)	12,083 / 17,011 (963)	2,189 / 3,713 (11)	4,353 / 8,238 (77)	2,235 / 4,608 (46)	**Pattern A が完了最速**
Tier 2 平均（3問）	3/3 一致	2,277 / 12,830 (0)	12,203 / 23,181 (1054)	2,844 / 4,991 (15)	4,963 / 14,174 (48)	2,895 / 9,320 (52)	**Pattern A が完了最速**
Tier 1–2 平均（6問）	6/6 一致	2,231 / 9,791 (9)	12,143 / 20,096 (1009)	2,517 / 4,352 (13)	4,658 / 11,206 (62)	2,565 / 6,964 (49)	**Pattern A が完了最速**
Tier 3 平均（3問）	3/3 一致	2,002 / 10,404 (0)	14,193 / 23,228 (1548)	2,131 / 21,580 (1056)	18,618 / 27,611 (1749)	3,659 / 23,685 (1396)	**Base Low は T3 で思考 0 tok！**
全9問平均	9/9 一致	2,155 / 9,995 (6)	12,826 / 21,140 (1188)	2,388 / 10,094 (361)	9,311 / 16,674 (624)	2,930 / 12,538 (498)	**Pattern A が総合最適**

注目すべき発見（なぜ固定 `thinking=low` では不十分で、かつ Pattern A が Pattern C より速いのか）：

1. **固定 `base_low` は Tier 3 全3問で内部思考トークンが `0 tok` だった**：`gemini-3.8-flash` に `thinking_level=LOW` を指定して通常の質問応答を行わせた結果、`T3_1`・`T3_2`・`T3_3` のいずれの複雑計算でも `thoughts_token_count = 0` となり、事前検算なしに即答を開始した。つまり複雑計算で確実に内部スクラッチパッド検算を発動させるには、Stage 2 として `medium` または `high`（本実験では `high`、平均 `1,056〜1,548 tok`）へエスカレーションする仕組みが不可欠である。
2. **Pattern A は Tier 3 で Pattern C より `1.53 秒` 速く初動を開始する**：Pattern A の Stage 1 は Pydantic スキーマ（`{thinking: bool, response: str}`）の制約付きデコーディングにより、Tier 3 の全3問でわずか `0〜83 tok` の思考で即座に `"thinking": true` を出力し、平均 `2,131 ms` で受付1文を表示した（`Base High` の `14,193 ms` 比で **約12.1秒のフリーズ解消**）。

4.4 ストリーム軌跡の深掘り（`thinking=false` 直接完結 vs. `thinking=true` 並列チェイニング）

Pattern A がどのようにして「定型質問での単一ストリーム完結（`low`）」と「複雑計算での2段階並列チェイニング（`low` → `high`）」を切り替えているのかを視覚化するため、`load_adaptive_trajectory()` を用いて `T2_1`（**Tier 2：電源が入らない時の放電手順**）と `T3_1`（**Tier 3：PC修理 vs. 買い替え 3年間総コスト比較**）の累積出力文字数軌跡を比較したのが図4-4である。

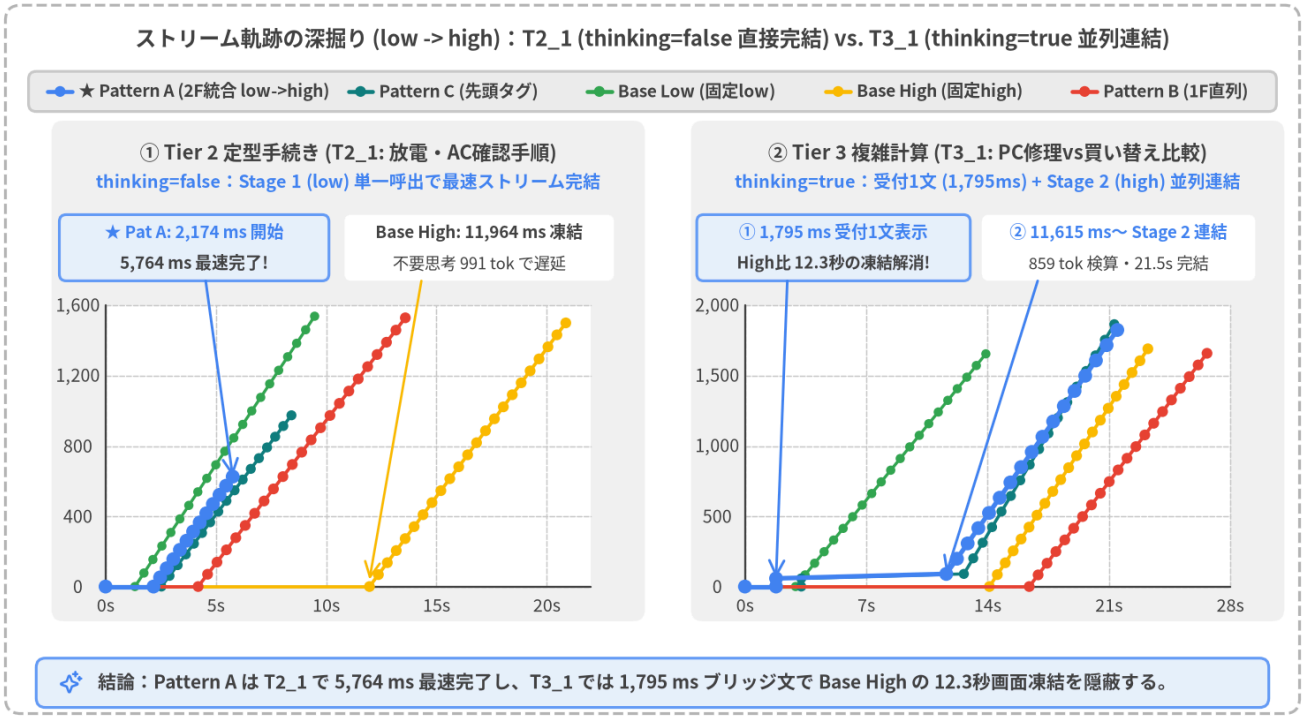


図4-4: Tier 2 定型手続き (T2_1: thinking=false 単一 low 完結) と Tier 3 複雑計算 (T3_1: thinking=true 受付1文 + Stage 2 high 並列連結) のストリーム軌跡比較

図4-4 の軌跡が示す2つの決定的な優位性

1. **左パネル `T2_1`（`thinking=false`：電源が入らない時の放電手順）における単一 API 完結と最速完了：**
   - Pattern A（青太線）は `2,174 ms` で `"thinking": false` を確定して `"response"` フィールドのストリーミングを開始し、`5,764 ms` **でストリーム完了** して全5方式で最も早く

回答を完結させている。

- 対して Base High（黄線）は不要な内部推論を `991 tok` も回すため `11,964 ms` **まで1文字も表示されず**、完了に `20,864 ms` を要している。

2. **右パネル `T3_1`（`thinking=true`：PC修理 vs. 買い替え3年比較）における `12.3 秒` の画面凍結解消：**
   - Base High（黄線）は `1,466 tok` の内部推論が終わる `14,104 ms` **にわたり累積文字数 0 文字のまま画面が凍結** し、Pattern B（赤線）はさらに遅い `16,402 ms` まで1文字も表示されない。また Base Low（緑線）は `2,922 ms` で始まるものの内部推論が `0 tok` で走っており事前検算が行われない。
   - 対して Pattern A（青太線）は、`1,795 ms` で `"thinking": true` を検知した瞬間にバックグラウンドで Stage 2（`thinking=high`）を非同期起動しつつ、Stage 1 のブリッジメッセージ（58文字）を即座に画面へ表示する。これにより、Base High と比較して `12,309 ms`（**約 `12.3 秒`）もの無応答フリーズを完全に解消** している。ユーザーがブリッジ文を読んでいる間に Stage 2 が `859 トークン` の内部推論を終え、`11,615 ms` から本計算のストリームが滑らかに立ち上がって `21,482 ms` に正確な試算結果（修理 `162,400円` vs. 買い替え `184,500円`、**修理が `22,100円` お得**）を完結させている。

4.5 Google AI Studio 実装コードと総合評価スコアカード

Pattern A の Google AI Studio（`google-genai`）リファレンス実装（`low` → `high`）

`google-genai` SDK のストリーミング時は `chunk.parsed` がストリーム途中で常に `None` となるため、Pydantic v2 標準同梱の `pydantic_core.from_json(..., allow_partial="trailing-strings")` を用いて不完全な JSON バッファから安全に `thinking` フラグとエスケープ解決済みの `response` デルタを抽出する（`src/gemini_evaluation/adaptive_thinking.py`）。なお、4.1節で述べたとおり、初期応答をさらに高速化したければ Stage 1 の `STAGE1_MODEL` を `"gemini-3.5-flash-lite"`（`MINIMAL` または `LOW`）に変更し、Stage 2 の思考をより深くしたければ `STAGE2_MODEL` を `"gemini-4-hydrogen"`（または複雑業務システムでは `"gemini-4-argon"` / `"gemini-4-carbon"`）へ差し替えるだけでそのまま動作する。

```python
import asyncio
from collections.abc import AsyncIterator
from contextlib import aclosing
from typing import Any

from google import genai
from google.genai import types
from pydantic import BaseModel
from pydantic_core import from_json

# 標準構成: gemini-3.8-flash (Stage 1: LOW -> Stage 2: HIGH)
#  ・初期応答をさらに高速化する場合: STAGE1_MODEL = "gemini-3.5-flash-lite"
#  ・思考をより深くする場合: STAGE2_MODEL = "gemini-4-hydrogen" (または "gemini-4-argon" / "gemini-4-carbon")
STAGE1_MODEL = "gemini-3.8-flash"
STAGE2_MODEL = "gemini-3.8-flash"


class AdaptiveStreamResponse(BaseModel):
    """Pattern A 用の最小2フィールド構造化ストリーミングスキーマ。"""

    thinking: bool  # 第1フィールド：初回チャンクで確定するルーティングフラグ
    response: str   # 第2フィールド：thinking=False 時は回答全文、True 時は受付1文


def parse_partial_json_dict(raw_buf: str) -> dict[str, Any] | None:
    """ストリーム途中の不完全な JSON を例外なく安全に辞書へ逐次パースする。"""
    if not raw_buf:
        return None
    try:
        parsed = from_json(raw_buf.encode("utf-8"), allow_partial="trailing-strings")
        return parsed if isinstance(parsed, dict) else None
    except ValueError:
        return None


async def stream_adaptive_router(
    client: genai.Client,
    prompt: str,
) -> AsyncIterator[str]:
    """Pattern A: 2フィールド統合 Pydantic ストリーム (low) + Stage 2 並列チェイニング (high)。"""
    stage2_queue: asyncio.Queue[str | BaseException | None] = asyncio.Queue()
    stage2_task: asyncio.Task[None] | None = None

    async def _produce_stage2() -> None:
        try:
            st2 = await client.aio.models.generate_content_stream(
                model=STAGE2_MODEL,
                contents=prompt,
                config=types.GenerateContentConfig(
                    system_instruction=(
                        "冒頭の挨拶や受付文は既に表示済みです。"
                        "挨拶を繰り返さず、直ちに本題の計算ステップと結論を回答してください。"
                    ),
                    thinking_config=types.ThinkingConfig(
                        thinking_level=types.ThinkingLevel.HIGH,
                    ),
                    temperature=0.2,
```

```
                ),
            )
            async with aclosing(st2):
                async for ch2 in st2:
                    if ch2.text:
                        await stage2_queue.put(ch2.text)
            await stage2_queue.put(None)
        except BaseException as exc:
            await stage2_queue.put(exc)

    json_buf = ""
    emitted_len = 0
    decided_thinking: bool | None = None

    try:
        st1 = await client.aio.models.generate_content_stream(
            model=STAGE1_MODEL,
            contents=prompt,
            config=types.GenerateContentConfig(
                system_instruction=(
                    "挨拶や定型手続き案内では thinking: false とし response に回答全文を出力してください。"
                    "複数条件の数値計算・比較では thinking: true とし response に1文の受付メッセージのみを出力してください。"
                ),
                response_mime_type="application/json",
                response_schema=AdaptiveStreamResponse,
                thinking_config=types.ThinkingConfig(
                    thinking_level=types.ThinkingLevel.LOW,
                ),
                temperature=0.2,
            ),
        )
        async with aclosing(st1):
            async for ch1 in st1:
                if not ch1.text:
                    continue
                json_buf += ch1.text
                partial = parse_partial_json_dict(json_buf)
                if partial is None:
                    continue

                # 1) thinking フラグが確定した瞬間に1度だけ判定し、True なら Stage 2 (HIGH) を並列起動
                if decided_thinking is None and isinstance(partial.get("thinking"), bool):
                    decided_thinking = partial["thinking"]
                    if decided_thinking:
                        stage2_task = asyncio.create_task(_produce_stage2())

                # 2) response の未送出デルタを即時 yield
                resp_so_far = partial.get("response")
                if isinstance(resp_so_far, str) and len(resp_so_far) > emitted_len:
                    delta = resp_so_far[emitted_len:]
                    emitted_len = len(resp_so_far)
                    yield delta

        # 3) EOF で Pydantic スキーマ完全検証を実行
        final_obj = AdaptiveStreamResponse.model_validate_json(json_buf)
        if len(final_obj.response) > emitted_len:
            yield final_obj.response[emitted_len:]
```

```
        # 4) thinking=True の場合は並列実行中の Stage 2 ストリームをシームレスに連結
        if decided_thinking and stage2_task is not None:
            yield "\n\n"
            while True:
                item = await stage2_queue.get()
                if item is None:
                    break
                if isinstance(item, BaseException):
                    raise item
                yield item
            await stage2_task
    finally:
        # 呼び出し側の中断や例外発生時に未完了の Stage 2 タスクを確実にキャンセル
        if stage2_task is not None and not stage2_task.done():
            stage2_task.cancel()
```

全5方式の総合評価スコアカード（`gemini-3.8-flash`: `low` → `high`）

評価項目	★ Pattern A (`pat_a_pydantic_2f`) `{thinking, response}` 統合 (`low->high`)	Pattern C (`pat_c_tag`) `[DIRECT]` / `[THINK]` タグ (`low->high`)	Pattern B (`pat_b_pydantic_1f`) `{thinking}` 直列ルーター (`low->high`)	Base High (`base_high`) 固定 `thinking=high`	Base Low (`base_low`) 固定 `thinking=low`
全9問平均 TTFT	`2,388 ms` (優良 `< 2.5s`)	`2,930 ms` (許容 `< 3.5s`)	`9,311 ms` (不可 `> 3.5s`)	`12,826 ms` (不可 `> 3.5s`)	`2,155 ms` (優良 `< 2.5s`)
Tier 1–2 平均 TTFT / 完了	`2,517 ms` / `4,352 ms` (最速完了)	`2,565 ms` / `6,964 ms`	`4,658 ms` / `11,206 ms`	`12,143 ms` / `20,096 ms`	`2,231 ms` / `9,791 ms`
Tier 3 平均 TTFT（最大）	`2,131 ms` (`2,728 ms`)	`3,659 ms` (`5,654 ms`)	`18,618 ms` (`24,231 ms`)	`14,193 ms` (`16,510 ms`)	`2,002 ms` (`2,922 ms`)
ルーティング正解率	`9 / 9` (`100%`)	`9 / 9` (`100%`)	`9 / 9` (`100%`)	固定（常時 `high`）	固定（常時 `low`）
Tier 3 複雑計算検証 (`ThTok`)	`3 / 3` (`674〜1,591 tok` 検算)	`3 / 3` (`806〜2,073 tok` 検算)	`3 / 3` (`1,362〜2,346 tok` 検算)	`3 / 3` (`1,295〜1,882 tok` 検算)	不可（全3問で `0 tok`・無検算）
API 呼び出し回数 (T1–2 / T3)	`1.0 回` / `2.0 回`	`1.0 回` / `2.0 回`	`2.0 回` / `2.0 回`	`1.0 回` / `1.0 回`	`1.0 回` / `1.0 回`
デコーダ層の型保証・拡張性	優良（Constrained Decoding）	不可（タグ露出・履歴ドリフト懸念）	一部のみ（ルーターのみ型保証）	なし（平文）	なし（平文）
総合判定	★ 本番標準採用（最速＆型安全）	不採用（A より遅く型保証なし）	不採用（最大 `24.2s` 凍結）	非推奨（最大 `16.5s` 凍結）	不採用（複雑計算で思考 `0 tok`）

5. マルチターンチャットでの挙動とコンテキストキャッシュ

実運用の対話型AIアプリケーション（パソコンのトラブルシューティング窓口やテクニカルサポート等）では、単発プロンプトではなく数十ターンにわたる連続対話が行われる。さらに実際のサポート対話では、大半を占める「定型的な確認・手順案内ターン」の合間に、バッテリー持続時間やクラウド転送時間、3年間の修理vs買い替え総費用（TCO）といった「多段階の条件計算・推論が必要なターン」が混在する。

本章では、すべて **Streaming（`generate_content_stream`）に統一** したうえで、Google AI Studio（`google-genai` SDK / `gemini-3.8-flash`）における **50ターン連続PCサポートチャットベンチマーク**（`src/gemini_evaluation/benchmark_multiturn_chat.py` / `multiturn_chat_log.json`、全50ターン × 3構成 ＝ 計150実行）の実測データに基づき、以下の **(1) 単一モデル固定 Streaming 2構成** と **(2) 第4章の Adaptive Thinking Router 構成** における **初期応答時間（`TTFT`）** と **全体完了時間（`Total`）**、および設定不要で自動発動する **暗黙的コンテキストキャッシュ（Implicit Context Caching：`cached_content_token_count`）** の実効性を定量的に解明する。

1. `(1)` **単一モデル固定 Streaming（`gemini-3.8-flash` `low` / `high`）**：
   - `streaming_low`（`3.8-flash [low]` **固定**）：全50ターンを `thinking_level="low"` でストリーミング応答。
   - `streaming_high`（`3.8-flash [high]` **固定**）：全50ターンを `thinking_level="high"` でストリーミング応答。
2. `(2)` **第4章の Adaptive Thinking Router 構成（`adaptive_pat_a`：Pattern A `low → high`）**：
   - Stage 1（`3.8-flash [low]`）で 2フィールド統合 Pydantic スキーマ `{thinking: bool, response: str}` をストリーミング生成し、定型ターン（`thinking=false`：40ターン）はそのまま1回の API 呼び出しで即答完走、複雑計算ターン（`thinking=true`：5ターンに1回の計10ターン）は1文のブリッジメッセージを即時ストリーミングしつつ裏で Stage 2（`3.8-flash [high]`）を並列起動して深層推論回答を後続結合する構成。

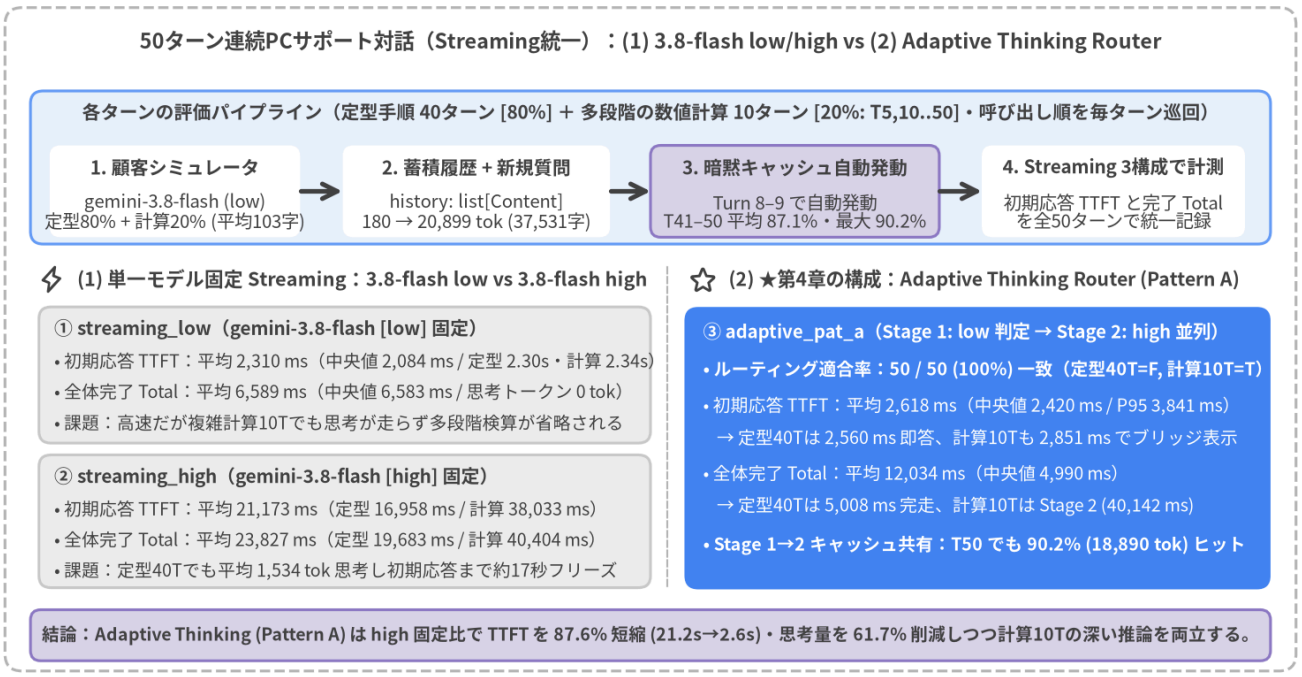


図5-1: 50ターン連続PCサポートチャット（Streaming統一）の評価アーキテクチャと3構成の初期応答（TTFT）・完了時間（Total）比較

5.1 マルチターン（50ターン連続）PCサポートチャットベンチマークの設計（Streaming統一）

1. `google-genai` におけるマルチターン履歴送信と Adaptive Thinking の結合

ステートレスな LLM API でマルチターン対話を実装する場合、アプリケーション側で過去の対話履歴 `history: list[types.Content]`（`role="user"` および `role="model"` の交互配列）を保持し、毎ターンのリクエスト時に過去の全履歴リストの末尾へ新規ユーザー発言を結合して `contents` へ渡す構造となる（`src/gemini_evaluation/llm_call.py` および `src/gemini_evaluation/adaptive_thinking.py`）。

```python
from google import genai
from google.genai import types

client = genai.Client(api_key=GEMINI_KEY)

# 過去の累積履歴 history: list[types.Content] の末尾に当ターンの新規質問を追加して送信
contents: list[types.Content] = [
    *history,
    types.Content(role="user", parts=[types.Part.from_text(text=user_prompt)]),
]

# (1) 単一モデル固定 Streaming (thinking_level="low" または "high")
response_stream = await client.aio.models.generate_content_stream(
    model="gemini-3.8-flash",
    contents=contents,
    config=types.GenerateContentConfig(
        system_instruction=AGENT_SYSTEM_INSTRUCTION,
        thinking_config=types.ThinkingConfig(thinking_level="low"),  # または "high"
        temperature=0.2,
    ),
)
```

本ベンチマーク（`src/gemini_evaluation/benchmark_multiturn_chat.py`）では、**「パソコンが突然動かなくなった顧客とテクニカルサポート窓口の段階的トラブルシューティング会話」** を模した50ターン連続シナリオにおいて、実運用と同じく **定型案内ターン（80%：40ターン）** と **多段階の数値計算・比較推論ターン（20%：5ターンに1回の計10ターン）** を織り交ぜて計測を実施した。

1. **顧客シミュレータ（User Simulator：`gemini-3.8-flash`, `thinking_level="low"`）**：Turn 1〜50 の段階的シナリオに沿って顧客質問（実測平均 `103.4 文字`）を生成する。
    - **定型確認・手順案内ターン（40ターン：`expected_thinking=False`）**：電源・ACアダプタ確認、30秒放電、BIOS確認、セーフモード起動、`sfc /scannow`、クリーンブート、BitLocker回復キー確認、梱包手順、返却後チェックなど、即座に手順を案内すべき40ターン。
    - **複雑計算・多段階推論ターン（10ターン：`expected_thinking=True`、`Turn 5, 10, 15, 20, 25, 30, 35, 40, 45, 50`）**：各10ターン区間に2回ずつ（5の倍数ターン）、具体的な数値条件を提示して正確な段階的計算を求める質問（例：Turn 5 の停電時バッテリー残量 `36Wh`・消費電力 `45W` での `18GB` 外付けSSD退避可否計算、Turn 25 の `32GB RAM` 環境でのページングファイル・休止状態 `hiberfil.sys` 占有容量試算、Turn 40 の自然故障無償修理 vs 落下物損時の自己負担額比較、Turn 50 の3年間クラウド＋外付けSSDバックアップ総費用と月間稼働率99.5%許容ダウンタイム計算など）を配置。
2. **Streaming 3構成の同一履歴・巡回計測（`rot = (turn - 1) % 3`）**：Turn 1（`180 prompt_tokens`）から Turn 50（`20,899 prompt_tokens`、履歴累計 `37,531 文字`）までの各ターンにおいて、同一の `history` に対して以下の3構成をすべて **Streaming**

（`generate_content_stream`） で実行し、呼び出し順を毎ターン巡回した（次ターンへ引き継ぐ正準履歴には `streaming_low` の回答平均 `647.2 文字` を採用）。

比較構成（すべて Streaming）	Stage 1 / 単一構成	Stage 2（`thinking=true` 時）	初期応答 `TTFT` 平均（中央値 / P95）	全体完了 `Total` 平均（中央値 / P95）	平均思考トークン数	定型40T / 計算10T の挙動と特徴
(1) `streaming_low`	`3.8-flash [low]`	—（単一呼び出し）	`2,310 ms`（`2,084` / `4,269 ms`）	`6,589 ms`（`6,583` / `8,397 ms`）	`0 tok`	全ターン高速だが、計算10Tでも思考 `0 tok` となり多段階検算を行わない
(1) `streaming_high`	`3.8-flash [high]`	—（単一呼び出し）	`21,173 ms`（`22,012` / `41,657 ms`）	`23,827 ms`（`23,887` / `43,621 ms`）	`1,981 tok`	計算10Tで深い推論（`3,772 tok`）を行うが、定型40Tでも `1,534 tok` 思考し初期応答に約17秒要する
(2) ★`adaptive_pat_a`（第4章 Pattern A）	`3.8-flash [low]`（`{thinking, response}`）	`3.8-flash [high]`（並列起動・キャッシュ共有）	`2,618 ms`（`2,420` / `3,841 ms`）	`12,034 ms`（`4,990` / `43,215 ms`）	`758 tok`（`-61.7%`）	**適合率** `50/50 (100%)`：定型40Tは `2,560 / 5,008 ms` で即答、計算10Tは `2,851 ms` でブリッジ表示＋裏で `high` 推論（`40,142 ms`）

5.2 入力トークン数の直線的増加と暗黙的コンテキストキャッシュ（Stage 1・Stage 2 間のキャッシュ共有）

50ターンの連続対話において、各ターン完了後に `(user, model)` の2メッセージを `history` に追加していくと、エージェントへの入力トークン数（`prompt_token_count`）は **1ターンあたり平均** `+423 tokens` のペースで直線的に増加する（Turn 1 で `180 tokens` → Turn 8 で `2,710 tokens` → Turn 25 で `9,523 tokens` → Turn 50 で `20,899 tokens`）。

本検証における重要な発見は、**入力コンテキストが約 `2,710〜3,050 tokens` に達した Turn 8〜9 において、明示的なキャッシュ作成 API を一切呼び出していないにもかかわらず、Gemini 3.8 Flash の暗黙的コンテキストキャッシュ（Implicit Context Caching：`usage_metadata.cached_content_token_count`）が自動的に発動し、さらに `adaptive_pat_a` の複雑計算ターンでは Stage 1（`low`）と Stage 2（`high`）の間でも同一履歴プレフィックスの KV キャッシュがそのまま共有された** 点である。

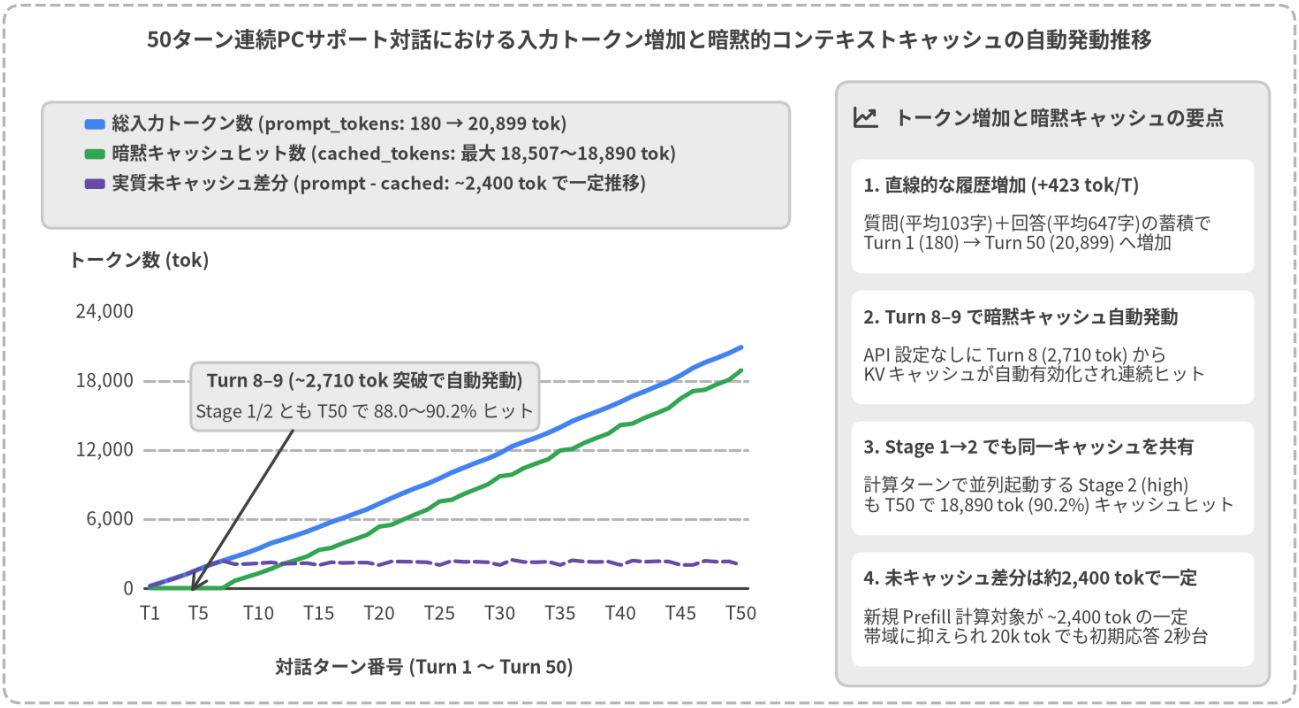


図5-2: 50ターン連続PCサポート対話における入力トークン数（prompt_tokens）の増加と暗黙的コンテキストキャッシュ（cached_tokens）のヒット推移

図5-2 から導かれる暗黙的コンテキストキャッシュと Adaptive Router の親和性

1. **Turn 8〜9 におけるゼロ設定発動と連続拡張**： Turn 1〜7（`180〜2,369 prompt_tokens`）では `cached_content_token_count = 0` であったが、入力が `2,710 tokens` に達した **Turn 8**（`adaptive_pat_a` で `633 cached_tokens`）および **Turn 9**（`3,050 tokens`、`streaming_low` で `728 cached_tokens`・`adaptive_pat_a` で `956 cached_tokens`）においてバックエンドの KV キャッシュが自動生成された。以降 Turn 50 まで毎ターンキャッシュ境界が前進し、**Turn 41〜50 の平均キャッシュヒット率は** `87.1%〜87.7%`**（Turn 50 単体では** `88.0%〜90.2%`**）** に達している。
2. **Adaptive Thinking Router（Pattern A）における Stage 1 → Stage 2 キャッシュ共有効果**： `adaptive_pat_a` では、Stage 2（`thinking_level="high"`）の `system_instruction` をチャット本体の `AGENT_SYSTEM_INSTRUCTION` と完全一致させ、Stage 1 のルーティング指示は末尾のユーザーメッセージ側に付与するプレフィックス不変設計（Prefix-Preserving Prompting）を採用している。その結果、複雑計算ターン（`Turn 10, 20, 25, 30, 40, 50`）で Stage 2 が起動した際にも、Stage 1 とまったく同じ過去履歴 KV キャッシュ（Turn 50 の Stage 1 で `18,507 tok`、**Stage 2 で** `18,890 tok [90.2%]`）が即座にヒットし、Stage 2 起動時の Prefill 再計算コストが最小化されている。

5.3 全50ターンにわたる初期応答（TTFT）と完了時間（Total）の推移比較

グラフの視認性を高め、`(1) 3.8-flash low / high` と `(2) Adaptive Thinking Router (Pattern A)` の3構成の違いを明確に比較するため、図5-3では **左パネルに「初期応答時間（`TTFT`）の50ターン推移」** を、**右パネルに「全体完了時間（`Total`）の50ターン推移」** を分けてプロットした。

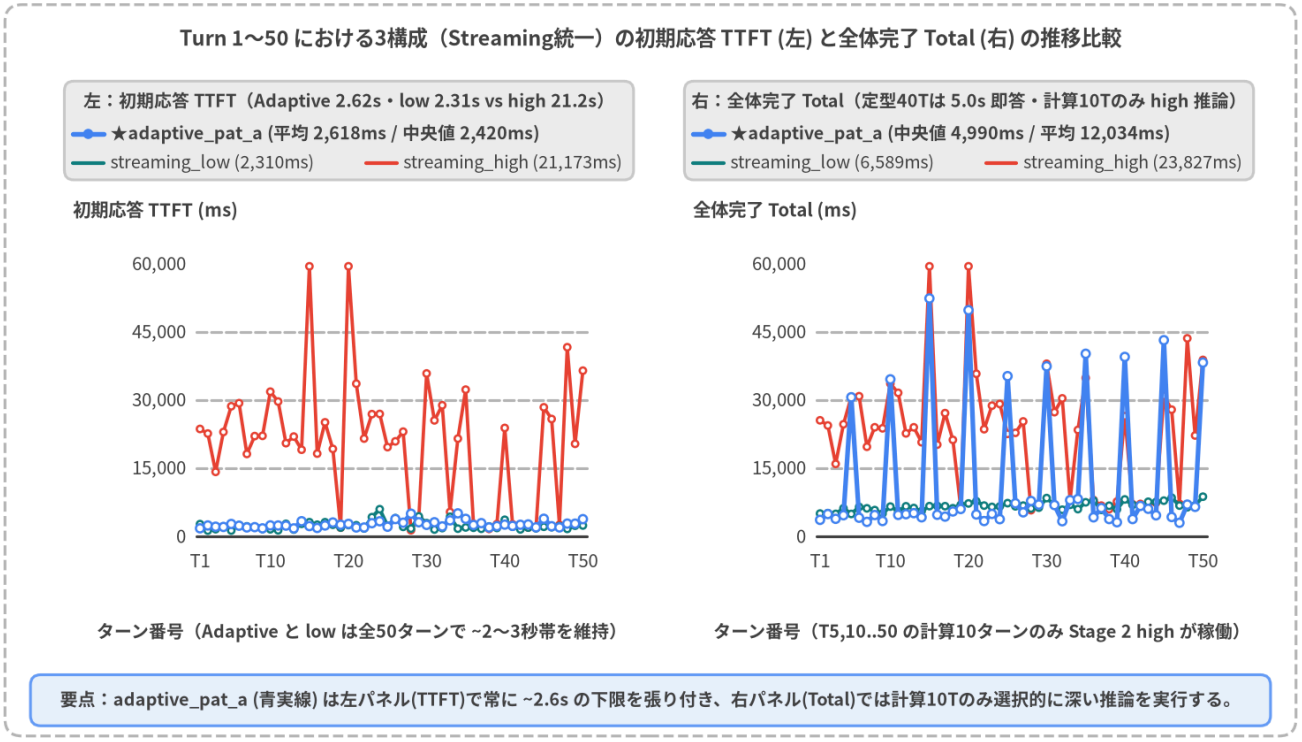


図5-3: Turn 1〜50 における3構成のレイテンシ推移（左パネル：初期応答 TTFT 比較、右パネル：全体完了 Total 比較）

図5-3 の左右比較から読み取れる重要事実

1. **左パネル（初期応答 `TTFT`）：`adaptive_pat_a` と `streaming_low` は全50ターンで `2〜3秒台` に完全収束：**
   - 赤線（`streaming_high`）は定型確認ターンでも内部思考（平均 `1,534 tok`）のため平均 `16,958 ms`、複雑計算ターンでは平均 `38,033 ms`（全50ターン平均 `21,173 ms`）まで1文字目が出力されない。

- 対して青線（`adaptive_pat_a`、全50ターン平均 `2,618 ms`、中央値 `2,420 ms`、P95 `3,841 ms`）は、緑線（`streaming_low`、平均 `2,310 ms`、中央値 `2,084 ms`）とほぼ完全に重なってグラフ最下部の `2〜3秒帯` をフラットに推移しており、複雑計算ターン（`T5, 10, ..., 50`）においても `2,851 ms` で1文目のブリッジメッセージを即座にストリーミング開始できている。

2. **右パネル（全体完了 `Total`）：定型40ターンの即答完走（`5.0秒`）と計算10ターンの選択的深層推論（`40.1秒`）：**
   - 定型案内40ターン（`expected_thinking=False`）において、`adaptive_pat_a` は Stage 1（`low`）の1回呼び出しのみで完結し、簡潔な構造化応答によって平均 `5,008 ms`（**中央値** `4,695 ms`）で完了する（`streaming_low` の平均 `6,411 ms` よりも高速）。
   - そして複雑計算10ターン（`Turn 5, 10, 15, 20, 25, 30, 35, 40, 45, 50`）においてのみ、`adaptive_pat_a` の青線が選択的に立ち上がり、Stage 2（`high`、平均 `3,671 思考トークン`）による厳密な多段階計算を実行して平均 `40,142 ms`（`streaming_high` の平均 `40,404 ms` と同等）で完了する。結果として、50ターン全体の中央値完了時間は `4,990 ms`（`streaming_high` 中央値 `23,887 ms` 比で `-79.1%` **短縮**）を達成している。

5.4 思考トークン分布の二極化と Turn 1・25・50 のストリーム軌跡比較

3構成における内部思考トークン数（`thoughts_token_count`）と完了時間（`Total`）の関係、および会話序盤（**Turn 1**：定型）・中盤（**Turn 25**：複雑計算）・終盤（**Turn 50**：複雑計算）におけるチャンク送出軌跡の違いを同時検証するため、図5-4の左パネルに全150実行（50ターン × 3構成）の `thoughts_tokens` **対** `total_ms` **散布図** を、右パネルに `load_multiturn_trajectory()` から抽出した **Turn 1・Turn 25・Turn 50 のストリーム軌跡オーバーレイ** を示す。

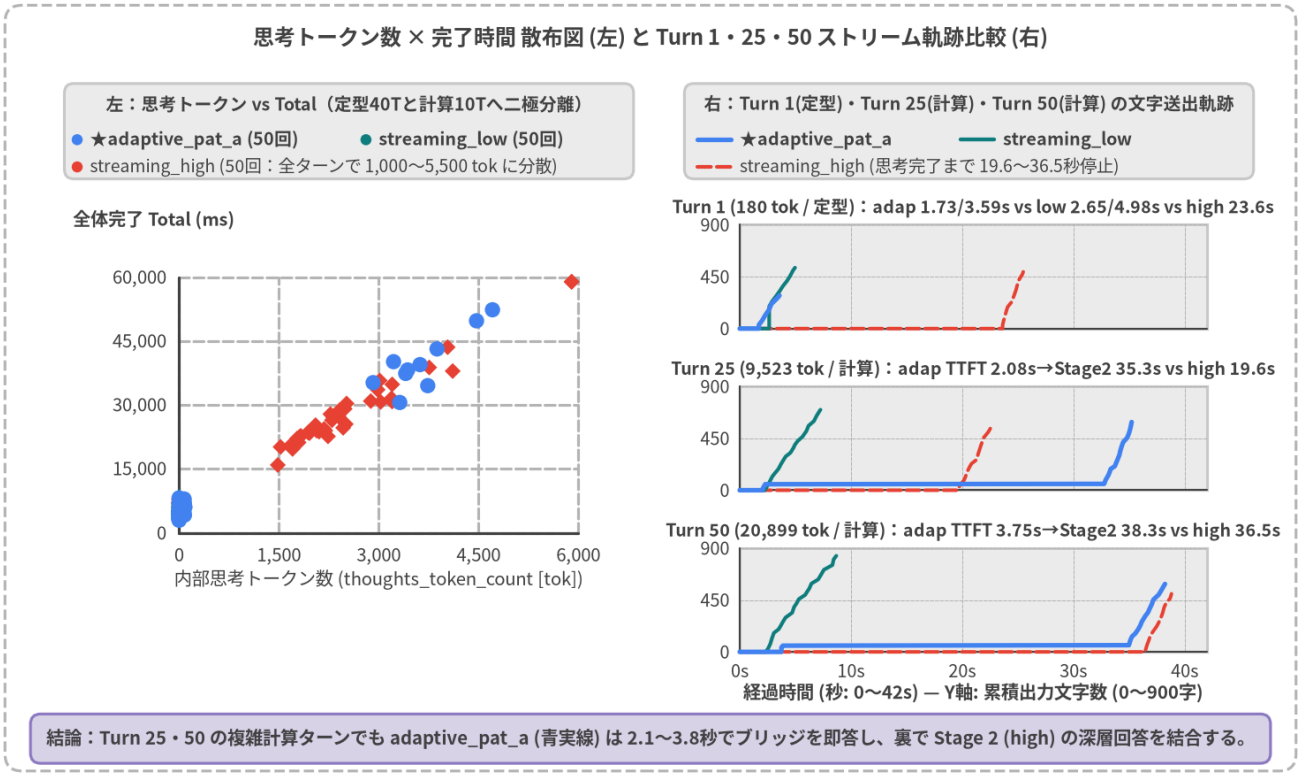


図5-4: 3構成の思考トークン数と完了時間の散布図（左）および Turn 1・25・50 におけるストリーム軌跡比較（右）

図5-4 から導かれる2つの定量的結論

1. **左パネル：`adaptive_pat_a` による「定型40ターンのゼロ思考クラスタ」と「計算10ターンの深層推論クラスタ」への完全二極化：**
   - `streaming_high`（赤ひし形）は定型ターンを含め全50ターンで `1,000〜5,500 tokens` の思考が発生し、完了時間が `10〜60秒` に広く分散している。
   - 一方 `adaptive_pat_a`（青丸）は、左下の **「定型40ターン群（思考 `~0〜30 tok`・完了平均 `5,008 ms`）」** と右上の **「複雑計算10ターン群（思考平均 `3,671 tok`・完了平均 `40,142 ms`）」** の2つのクラスタへ綺麗に二極分離しており、不要な思考コストを `61.7%` **削減** している。
2. **右パネル：Turn 1・Turn 25・Turn 50 を通じた2段階ストリーム軌跡の比較：**
   - **Turn 1（`180 tok`・定型起動相談・`expected_thinking=False`）：** `adaptive_pat_a`（青実線）は `1,726 ms` でストリームを開始して `3,592 ms` で完走し、`streaming_low`（`2,652 / 4,984 ms`）と同等以上の速さで即答している（一方 `streaming_high` は `2,505 思考トークン` のため `23,636 ms` まで無応答）。

- **Turn 25（`9,523 tok`・キャッシュ `74.1%`・`32GB RAM` 環境でのページング/休止ファイル容量計算・`expected_thinking=True`）**：`adaptive_pat_a`（青実線）は `2,075 ms` で Stage 1 の1文ブリッジを送出して体感フリーズを解消し、同時に並列起動した Stage 2（`high`、キャッシュ `7,521 tok [77.1%]` **ヒット**、`2,916 思考トークン`）の計算結果を後続ストリームとして滑らかに結合している。
- **Turn 50（`20,899 tok`・キャッシュ `88.0%〜90.2%`・3年間バックアップ総費用＆稼働率 99.5%計算・`expected_thinking=True`）**：入力が2万トークンを超えた最終ターンでも、`adaptive_pat_a` は `3,753 ms` で初回チャンクを送出し（`streaming_high` の `36,492 ms` 比で `-32.7秒 [-89.7%]` **先行**）、Stage 2 でも暗黙キャッシュが `18,890 tok [90.2%]` ヒットすることで `streaming_high`（`38,857 ms`）と同等の `38,274 ms` で厳密な計算回答を完走している。

5.5 区間別・ターン種別パフォーマンス集計表とマルチターン本番設計ガイドライン

1. ターン種別（定型40ターン vs 複雑計算10ターン）および10ターン区間別の包括的統計表

全50ターンを「定型案内ターン（40ターン）」「複雑計算ターン（10ターン：`T5, 10, ..., 50`）」および10ターンごとの5コホート（`Turns 1–10` 〜 `Turns 41–50`）で集計したベンチマーク結果を下表に示す。

集計区分（各コホートに定型8T＋計算2T）	平均入力（`prompt`）/ キャッシュ（ヒット率）	(1) `streaming_low` 初期応答 `TTFT` / 完了 `Total` [思考]	(1) `streaming_high` 初期応答 `TTFT` / 完了 `Total` [思考]	(2) ★`adaptive_pat_a` (Pattern A) 初期応答 `TTFT` / 完了 `Total` [思考]	`adaptive` vs `high` TTFT 短縮 / Total 短縮
Turns 1–10 (電源・放電・T5停電退避・T10復旧時間計算)	`1,796 tok`（`180〜3,438`） Cache: `179〜287 tok`（`10〜15%`）	`1,854` / `5,583 ms` [`0 tok`]	`23,553` / `25,329 ms` [`2,386 tok`]	`2,121` / `9,729 ms`（中央値 `4,332`）[`715 tok` / 適合 `10/10`]	`-21,432 ms`（`-91.0%`） Total: `-61.6%`
Turns 11–20 (ドライバ・T15転送・T20再構築時間計算)	`5,533 tok`（`3,893〜7,334`） Cache: `3,151〜3,383 tok`（`57〜60%`）	`2,372` / `6,396 ms` [`0 tok`]	`29,886` / `32,175 ms` [`2,846 tok`]	`2,431` / `14,142 ms`（中央値 `4,953`）[`949 tok` / 適合 `10/10`]	`-27,455 ms`（`-91.9%`） Total: `-56.0%`

集計区分（各コホートに定型8T＋計算2T）	平均入力（prompt）/ キャッシュ（ヒット率）	(1) streaming_low 初期応答 TTFT / 完了 Total [思考]	(1) streaming_high 初期応答 TTFT / 完了 Total [思考]	(2) ★adaptive_pat_a (Pattern A) 初期応答 TTFT / 完了 Total [思考]	adaptive vs high TTFT 短縮 / Total 短縮
Turns 21–30 (復元・T25仮想メモリ・T30帯域計算)	9,759 tok (7,800〜11,710) Cache: 7,321〜7,530 tok (75〜76%)	3,143 / 6,880 ms [0 tok]	21,374 / 23,842 ms [1,965 tok]	2,926 / 11,670 ms (中央値 6,104) [652 tok / 適合 10/10]	-18,448 ms (-86.3%) Total: -51.1%
Turns 31–40 (暗号化・T35読取・T40保証自己負担計算)	14,201 tok (12,293〜16,156) Cache: 11,752〜11,947 tok (83〜84%)	2,212 / 6,737 ms [0 tok]	14,686 / 17,870 ms [1,237 tok]	2,957 / 12,319 ms (中央値 6,530) [716 tok / 適合 10/10]	-11,729 ms (-79.9%) Total: -31.1%
Turns 41–50 (修理・T45 3年TCO・T50保守費用計算)	18,757 tok (16,661〜20,899) Cache: 16,337〜16,528 tok (87〜88%)	1,970 / 7,348 ms [0 tok]	16,365 / 19,921 ms [1,473 tok]	2,655 / 12,312 ms (中央値 6,242) [757 tok / 適合 10/10]	-13,710 ms (-83.8%) Total: -38.2%
定型案内 40ターン平均 (expected_thinking=False)	9,676 tok (1回呼び出し完結)	2,304 / 6,411 ms (中央値 2,060 / 6,497) [0]	16,958 / 19,683 ms (中央値 20,701 / 22,721) [1,534]	2,560 / 5,008 ms (中央値 2,352 / 4,695) [30]	-14,398 ms (-84.9%) Total: -74.6%
複雑計算 10ターン平均 (expected_thinking=True: T5..50)	11,341 tok (Stage 1 + Stage 2 並列)	2,336 / 7,302 ms (中央値 2,310 / 7,370) [0]	38,033 / 40,404 ms (中央値 32,083 / 34,257) [3,772]	2,851 / 40,142 ms (中央値 2,618 / 38,900) [3,671]	-35,182 ms (-92.5%) Total: -0.6% (同等推論)
全50ターン総合平均 (中央値 / P95)	10,009 tok (180〜20,899) Cache: 77.4%〜78.5% (最大 ** 90.2% **)	2,310 / 6,589 ms (中央値 2,084 / 6,583) [0]	21,173 / 23,827 ms (中央値 22,012 / 23,887) [1,981]	2,618 / 12,034 ms (中央値 2,420 / 4,990) [758]	-18,555 ms (-87.6%) Total: -49.5% (中央値 -79.1%)

2. マルチターンチャット本番設計の3大ベストプラクティス

1. **プレフィックス不変性（Prefix Immutability）の厳守と Stage 1・Stage 2 キャッシュ共有**：
   - Gemini 3.8 Flash の暗黙的コンテキストキャッシュは、`system_instruction` と過去の `history: list[types.Content]` の **先頭からのトークン系列が完全一致（Prefix Match）** している場合に自動で KV キャッシュを再利用する。
   - **推奨パターン**：`system_instruction` と過去ターンの `history` は完全な **追記専用（Append-Only）** として不変に保ち、Adaptive Thinking Router の Stage 1 と Stage 2 でも同一の `system_instruction` を共有する（Stage 1 固有の JSON 出力指示は末尾の最新ユーザー発言側に結合する）。これにより、Stage 2（`high`）が並列起動した際にも過去履歴の最大 `90.2%`（**Turn 50 で** `18,890 tokens`）が即座にキャッシュヒットする。

2. **単一 `low` 固定と単一 `high` 固定のジレンマを第4章の Adaptive Thinking Router（Pattern A）で解消：**
   - 50ターンのサポート対話において、`streaming_low` 固定では初期応答（`2,310 ms`）・完了（`6,589 ms`）とも高速だが、バッテリー残量計算や3年間TCO試算などの複雑ターンでも思考トークンが `0 tok` となり計算ミスや条件見落としのリスクが残る。逆に `streaming_high` 固定では定型40ターンでも平均 `1,534 tok` の思考が走り、毎ターン1文字目が出るまで平均 `21.2 秒`（複雑ターンでは `38.0 秒`）画面がフリーズする。
   - 第4章で確立した **Pattern A（2フィールド統合構造化ストリーム：`low → high`）** を適用することで、**全50ターンで `2.62 秒`（中央値 `2.42 秒`）の初期応答** を維持しながら、定型40ターンは平均 `5.01 秒` で即答完走し、複雑計算10ターンのみ裏で `high` 推論（平均 `3,671 tok`）を走らせて高い計算精度を担保できる。
3. **さらなる初期応答高速化（`3.5-flash-lite`）や超深層推論（`Gemini 4` 系）への拡張：**
   - 第4章 4.1節で整理した通り、Stage 1 の初期応答（TTFT）を `2.6秒` からさらに `1.0〜1.5秒台` へ短縮したい場合は Stage 1 に `gemini-3.5-flash-lite`（`minimal` / `low`）を採用し、Stage 2 の思考をより深くしたい複雑な業務システムでは Stage 2 に `gemini-4-hydrogen [high]` や `gemini-4-carbon [medium/high]` を組み合わせる構成が推奨される。